

축구 영상 선수 기여도 분석 프로그램 완전 해설

16개 파일 전부가 무엇을 하는지, 프레임 하나가 좌표 한 줄이 되기까지,
그 좌표에서 기여도(SC · PR · PA · DPI · TC)가 계산되는 방식,
그리고 두 단계 사이에서 지금 무엇이 끊어져 있는지.

1판과의 차이 — 1판은 5개 파일만 보고 **기본값** 기준으로 썼다.

이번에는 16개 파일 전부와 **실제 config.json 설정값**을 반영했고,
tracker.py 를 끝까지 읽으면서 1판의 잘못 두 곳을 바로잡았다(8.4 참고).

작성일 · 2026-09-30

문서의 모든 수치는 코드에 적힌 실측값이거나, 코드를 직접 실행해 확인한 값이다.

목 차

이 문서의 구성

1. 한 장 요약

2. 전체 지도 — 16개 파일과 실행 사슬

2.1 두 단계 · 2.2 파일 16개 · 2.3 실행 사슬 · 2.4 데이터 흐름

3. 실행 계층 — 더블클릭에서 분석까지

3.1 SoccerTracker.exe · 3.2 START.bat · 3.3 ensure_python.ps1 · 3.4 bootstrap.py · 3.5 start.sh · 3.6 requirements.txt · 3.7 보조 배치와 아이콘

4. app.py — 사람이 만지는 부분

5. tracker.py (1) — 분석을 시작하기 전에

5.1 Config · 5.2 모델 준비와 OpenVINO · 5.3 트래커 설정 자동 생성 · 5.4 팀 색 보정 · 5.5 구간 분석과 워밍업

6. tracker.py (2) — 프레임 하나가 CSV 한 줄이 되기까지

6.1 검출 · 6.2 사람 필터 · 6.3 팀 판정 · 6.4 공 파이프라인 6단계 · 6.5 기록

7. tracker.py (3) — 분석이 끝난 뒤의 후처리

7.1 팀 다수결 · 7.2 골키퍼 트랙 · 7.3 불박이 제거 · 7.4 궤적 잇기 · 7.5 보간 · 7.6 저장 순서

8. 내 config.json 으로는 실제로 무엇이 켜져 있나 신규

8.1 설정 대조표 · 8.2 켜지지 않는 필터들 · 8.3 이 설정의 약점 · 8.4 1판의 정정

9. 결과를 읽는 법 — tracks.csv 와 summary.json

10. 기여도 계산 — SC · PR · PA · DPI · TC

10.1 무엇을 재나 · 10.2 계산 전 준비 · 10.3 공 소유 판정 · 10.4 SC · 10.5 PR · 10.6 PA · 10.7 합성 · 10.8 진단 출력

11. 두 단계 사이 — 끊어져 있는 여섯 곳

12. 사례 해부 — 결과표가 망가진 이유

13. 고치는 순서

14. 라이선스와 알려진 한계

15. 용어 사전

부록 A. tracks.csv 열 / B. summary.json 필드 / C. 상수 전체표

읽는 순서 제안

처음 읽는다면 1장 → 2장 → 8장 → 10장 → 12장. 이 다섯 장이면 프로그램이 무엇을 하고 지금 무엇이 문제인지 다 들어온다.

코드를 고칠 때 5~7장을 찾아보는 참고서로 쓰면 된다.

당장 결과가 필요하다면 11장 → 13장만 봐도 된다.

1 장

한 장 요약

이 프로그램이 하는 일

축구 경기 영상을 넣으면, 프레임마다 선수·골키퍼·심판·공이 화면 어디에 있는지를 찾아 CSV 한 장으로 뽑아 준다. 그게 전부다. 명령어를 칠 필요가 없도록 설치·실행·확인 도구가 함께 들어 있다.

기여도는 어디서 계산되나

핵심

16개 파일 안에는 기여도를 계산하는 코드가 없다. 전부 '측정' 단계다. 기여도(SC · PR · PA · DPI · TC)는 별도 파일 `soccer_dpi_pa_v7.py` 가 계산한다. 두 프로그램은 서로를 전혀 모르고, **tracks.csv 파일 하나**로만 이어진다. 그 이음매가 지금 어긋나 있어서(11장), 현재 상태로는 tracks.csv 를 기여도 코드에 그냥 넣을 수 없다.

지금 상태를 세 줄로

항목	상태	내용
검출·추적 품질	좋음	축구 전용 모델 + 공 전용 모델 + 6단계 공 오검출 필터. 페널티 마크·축구화·골대 기둥까지 실측으로 잡아내는 수준이다. 이 부분은 상용 도구에 견줄 만하다.
추적 ID 지속	문제	지금 설정(<code>vid_stride 3 · imgsz 640</code>)에서는 한 선수가 수십 개의 번호를 갈아탄다. 코드 주석의 실측값으로 트랙 지속 중앙값이 0.2~0.5초다. '선수별' 기여도 표가 무의미해지는 직접 원인.
좌표 단위	문제	CSV 좌표는 화면 픽셀이다. 미터로 바꾸는 호모그래피가 어디에도 구현되어 있지 않다. 기여도 수식의 상수가 전부 미터 기준이라, 이 상태로는 계산이 성립하지 않는다.

즉 측정 장비는 잘 만들어져 있는데, 눈금이 안 매겨져 있고 피사체에 이름표가 안 붙어 있는 상태다. 13장이 그 두 가지를 순서대로 고치는 방법이다.

2 장

전체 지도 — 16개 파일과 실행 사슬

2.1 두 단계

1단계 · 측정

영상 → 좌표

"이 프레임의 7번 선수는 화면 어디에 있나"

컴퓨터 비전 문제. YOLO 검출, ByteTrack 추적, 색 군집, 오검출 물리 필터.

파일 : 올려주신 16개 전부

산출 : results/<영상이름>/tracks.csv

2단계 · 해석

좌표 → 기여도

"그 자리에 서 있던 것이 팀에 얼마나 도움이 됐나"

축구 전술 + 통계 문제. 공간 분할, 협력게임 배분, 순위 정규화.

파일 : soccer_dpi_pa_v7.py (별도)

산출 : sc_pr_result.xlsx

2.2 파일 16개

파일	분류	역할
tracker.py 2,593행	측정 본체	이 프로그램의 전부. 검출·추적·필터·팀구분·공처리·후처리·CSV 기록. 터미널 단독 실행도 된다.
app.py 622행	GUI	tkinter 창. 대기열·프리스트·폴더감시·진행률·로그. 분석은 별도 스레드에서 tracker 를 부른다.
draw_boxes.py 402행	검증	CSV 를 원본 영상에 다시 그려 확인용 영상 생성. tracker 도 결과 영상을 만들 때 이 모듈을 부른다.
config.json 80개 항목	설정	모든 기준값. GUI 에서 바꾼 값이 여기 저장되고, CLI 도 이것을 읽는다.
recover_summary.py	복구	tracks.csv 는 있는데 summary.json 이 없는 결과 폴더를 되살린다.
README.md 209행	문서	설치·사용·문제해결·알려진 한계·라이선스. 제작자 본인의 한계 진술이 담겨 있어 중요하다(14장).
— 실행 계층 (3장) —		
SoccerTracker.exe	실행	더블클릭용 실행기. 창을 띄우는 것뿐이고 분석은 app.py 가 한다.
SoccerTracker.cs 129행	소스	위 exe 의 C# 원본. 윈도우에 기본 포함된 csc.exe 로 빌드한다.
START.bat	실행	윈도우 실행 스크립트. 파이썬을 세 단계로 찾고, 없으면 설치까지 간다.
ensure_python.ps1 121행	설치	파이썬이 없으면 3.12 를 사용자 영역에 조용히 설치한다(관리자 권한 불필요).

bootstrap.py 146행	설치	venv 를 만들고 GPU 유무에 따라 PyTorch·Ultralytics·lap 을 순서대로 설치한다.
start.sh 268행	실행	리눅스/맥용. venv 가 막힌 컨테이너에서는 libs/ 에 직접 설치로 자동 전환한다.
requirements.txt	참고	오프라인 설치·미러용 패키지 목록. 각 패키지가 왜 필요한지 주석이 달려 있다.
박스영상 만들기.bat	보조	draw_boxes.py 를 더블클릭으로 돌리는 배치.
exe 다시 만들기.bat	보조	아이콘을 새로 그리고 SoccerTracker.exe 를 다시 빌드한다.
make_icon.py 89행	보조	app.ico 를 코드로 그린다(Pillow). 잔디·공·추적 상자 모서리.

2.3 실행 사슬 — 더블클릭 한 번에 무슨 일이 일어나나

사용자가 SoccerTracker.exe 를 더블클릭 (또는 영상을 그 위에 끌어다 놓음)

SoccerTracker.exe — 쓸 만한 파이썬을 찾는다 (3단계)

① 이 폴더의 venv\Scripts\pythonw.exe → ② 시스템 파이썬 중 ultralytics·torch 가 이미 있는 것 → ③ 둘 다 없으면 START.bat 에 넘긴다

▼ (③인 경우)

START.bat — 같은 3단계를 배치로 한 번 더, 그리고 설치

파이썬이 아예 없으면 ensure_python.ps1 실행 → 경로를 _python_path.txt 로 받아 bootstrap.py 호출

ensure_python.ps1 → bootstrap.py — 최초 1회 환경 구축 (5~20분)

파이썬 3.12 사용자 설치 → venv 생성 → PyTorch(GPU 감지) → Ultralytics → lap → OpenVINO → tkinterdnd2

app.py — 창이 뜬다

GPU 유무를 뒤에서 확인해 프리셋을 자동 조정 · 영상 대기열 · [분석 시작]

tracker.run_batch() → analyze() — 실제 분석

프레임 루프 → tracks.csv → 후처리 → summary.json → 결과 영상(draw_boxes.py)

리눅스·맥에서는 이 사슬이 start.sh 하나로 압축된다. 화면이 없거나 tkinter 가 없으면 창 대신 터미널 모드로 tracker.py 를 직접 돌린다.

2.4 데이터 흐름

영상 (mp4)

videos/ 폴더 · 드래그앤드롭 · 파일 선택



프레임 루프 (6장)

YOLO 검출 → 잔디 밖 제외 → 상체 색으로 팀 → 공 전용 모델 + 확대 재검출 + 물리 검사 → CSV 한 줄



후처리 (7장)

팀 다수결 → 골키퍼 트랙 확정 → 불박이 공 제거 → 공 궤적 잇기 → 공 보간



results/<영상이름>/

tracks.csv (픽셀 좌표) · summary.json (통계) · <이름>_boxes.mp4 (확인용)



⚠ 끊어진 이음매 (11장)

픽셀↔미터 · 열 이름 · 팀 번호 규약 · 공 행 인식 · 보간 표시 · 심판 처리



soccer_dpi_pa_v7.py (10장)

공 소유 판정 → SC · PR · PA → DPI · TC · TC_adj → 엑셀

3 장

실행 계층 — 더블클릭에서 분석까지

이 계층의 파일들은 분석 품질과 직접 관계가 없어 보이지만, 실제로는 두 가지를 결정한다 — GPU 를 쓸 수 있는가, 그리고 추적 ID 가 제대로 붙는가. 둘 다 결과를 크게 바꾼다.

3.1 SoccerTracker.exe / SoccerTracker.cs — 검은 창 없이 띄우기

C# 129행짜리 실행기다. 윈도우에 기본으로 들어 있는 컴파일러(`csc.exe`)로 빌드하므로 빌드 도구를 따로 설치할 필요가 없다. 하는 일은 파이썬을 찾아 `app.py` 를 띄우는 것뿐이다.

순서	찾는 것	이유
①	<code>venv\Scripts\pythonw.exe</code>	이 폴더 전용 가상환경. <code>pythonw</code> (w 가 붙은 것)를 쓰면 검은 콘솔 창이 안 뜬다.
②	시스템 파이썬 중 <code>ultralytics-torch</code> 가 있는 것	이미 깔려 있으면 설치를 기다릴 이유가 없다. 확인은 <code>importlib.util.find_spec</code> 으로 한다 — 실제로 import 하지 않아서 빠르다 (<code>torch import</code> 는 몇 초 걸린다).
③	<code>START.bat</code>	아무것도 없으면 설치부터 해야 하고, 그건 진행 상황이 보여야 하므로 콘솔이 뜨는 배치로 넘긴다.

이 파일 위에 영상을 끌어다 놓으면 그 경로가 그대로 `app.py` 의 인자로 전달된다. 탐색 과정에는 30초 타임아웃이 걸려 있어, 응답 없는 파이썬 하나 때문에 프로그램이 멈추지 않는다.

3.2 START.bat — 같은 판단을 배치로

`exe` 없이도 똑같이 동작하도록 만든 대안 경로다. 순서는 `exe` 와 같고, 세 번째 단계에서 PowerShell 스크립트를 부른다.

```
powershell -NoProfile -ExecutionPolicy Bypass -File "ensure_python.ps1"
set "PYEXE="
if exist "_python_path.txt" set /p PYEXE=<"_python_path.txt"
del "_python_path.txt"
"%PYEXE%" "bootstrap.py" %*
```

PowerShell 이 찾아낸 파이썬 경로를 임시 텍스트 파일로 주고받는다. 배치와 PowerShell 사이에 값을 넘기는 가장 확실한 방법이고, 쓰고 나서 바로 지운다.

3.3 ensure_python.ps1 — 파이썬이 아예 없을 때

121행이지만 함정을 세 개나 피해 간다.

(1) Microsoft Store 업데이트 걸러내기

```
if ($Exe -like "*\WindowsApps\*") { return $false }
```

윈도우 10/11 에는 `python.exe` 라는 이름의 **0바이트 업데이트**가 기본으로 들어 있다. 실행하면 Microsoft Store 가 열릴 뿐이다. 이걸 파이썬으로 착각하면 설치가 무한히 실패한다.

(2) 진짜 파이썬인지 확인하는 방법

```
$ver = & $Exe -c "import sys,tkinter;print(sys.version_info[0]*100+sys.version_info[1])"
return ([int]$ver -ge 309) # 3.9 이상 + tkinter 존재
```

버전만 보지 않고 `tkinter` 를 함께 **import** 해 본다. GUI 가 필요하기 때문이다. tkinter 없이 설치된 파이썬은 나중에 창을 못 띄운다.

(3) 관리자 권한 없이 설치

```
/quiet InstallAllUsers=0 PrependPath=1 Include_tcltk=1 Include_pip=1
```

사용자 영역에 조용히 설치한다. 3.13 이 아니라 **3.12** 를 쓰는데, 최신 버전은 일부 패키지의 빌드된 휠이 아직 없을 수 있어서다. 직접 내려받기가 막힌 환경을 위해 `winget` 으로 한 번 더 시도한다.

3.4 bootstrap.py — 설치 순서가 데이터 품질을 결정한다

평범한 설치 스크립트처럼 보이지만, 여기 적힌 순서가 나중의 CSV 품질을 직접 결정한다.

순서 ① torch 를 ultralytics보다 먼저

```
gpu = has_nvidia_gpu() # nvidia-smi 가 도는지로 판단
if gpu: pip install torch torchvision --index-url .../whl/cu124
else: pip install torch torchvision
pip install ultralytics # ← 반드시 torch 다음
```

ultralytics 를 먼저 깔면 의존성 해결 과정에서 **CPU 전용 torch** 휠을 끌어온다. GPU 가 있는 PC 인데도 CPU 로 돌아 45분 영상이 몇 분 → 몇 시간이 된다.

순서 ② lap 을 미리

이것을 빠뜨리면 CSV 가 통째로 못 쓰게 된다

`lap` 은 ByteTrack 의 헝가리안 매칭 라이브러리다. 없으면 ultralytics 가 **첫 추적 도중에** 알아서 설치하는데, 그 **회차는 track_id 가 전부 -1 로 나온다.**

`track_id` 가 -1 이면 "같은 선수"라는 개념이 사라진다. 프레임별 좌표는 있지만 누가 누구인지 모르므로 기여도 계산의 선수별 집계가 원천적으로 불가능하다. 몇 시간을 돌리고도 버려야 하는 CSV 가 나온다. `requirements.txt` 에도 같은 경고가 주석으로 달려 있다.

순서 ③ 실패해도 되는 것과 아닌 것

OpenVINO·onnx·tkinterdnd2 는 실패해도 그냥 넘어간다(없어도 돌아간다). torch·ultralytics·lap 은 실패하면 **venv** 를 통째로 지우고 중단한다 — 반쯤 설치된 환경을 남기지 않기 위해서다.

3.5 start.sh — 리눅스, 그리고 권한이 없는 컨테이너

268행으로 가장 방어적인 스크립트다. Kasm 같은 **sudo** 도 **python3-venv** 도 없는 컨테이너를 염두에 두고 만들어졌다.

단계	내용
설치 0안 · 건너뛰기	<code>import torch, ultralytics, cv2</code> 가 이미 되면 설치를 통째로 건너뛴다.
설치 1안 · venv	표준 방식. <code>python3 -m venv</code> 가 실패하고 <code>sudo</code> 가 되면 <code>python3-venv</code> · <code>python3-tk</code> 설치를 시도한다.
설치 2안 · libs/	venv 가 안 되면 <code>pip install --target ./libs</code> 로 폴더에 직접 설치하고 실행할 때 <code>PYTHONPATH</code> 를 얻는다. 권한이 전혀 필요 없다.

libs/ 방식의 함정과 그 대비

`--target` 으로 깐 C 확장(numpy 등)은 **설치에 쓴 파이썬 버전에만** 맞는다. 다음 실행 때 다른 파이썬이 잡히면 numpy 부터 깨진다. 그래서 어느 파이썬으로 깔았는지를 `libs/.python` 과 `libs/.pyver` 에 적어 두고, 다음 실행 때 버전이 달라졌으면 **지우고 다시 설치한다.**

`bash start.sh --check` 는 진단만 하고 끝낸다 — OS·배포판·sudo 가능 여부·코어/메모리·디스크·DISPLAY·GPU·파이썬 버전·venv/tkinter/pip 유무·설치 상태·PyPI 연결까지 한 번에 찍어 준다. 문제를 물어볼 때 이 출력을 그대로 보내면 된다.

화면이 없거나(`DISPLAY` 없음) tkinter 가 없으면 **자동으로 터미널 모드**가 되어 `tracker.py` 를 직접 실행한다. 분석 결과는 창으로 돌릴 때와 똑같다.

3.6 requirements.txt — 왜 필요한지가 적힌 목록

패키지	분류	쓰이는 곳
torch / torchvision	핵심	YOLO 추론 엔진
ultralytics ≥8.3	핵심	YOLO11 + ByteTrack
lap ≥0.5	핵심	ByteTrack 헝가리안 매칭. 없으면 첫 회차 track_id 가 전부 -1
opencv-python ≥4.8	핵심	영상 디코딩, 잔디 마스크, 상체 색, 결과 영상
numpy ≥1.24	핵심	k-means, 마스크 연산
PyYAML ≥6.0	핵심	트래커 설정 자동 생성(5.3)

openvino / onnx	선택	CPU 추론 가속. 30분 영상 기준 20.3분 → 12.8분
tkinterdnd2	선택	창에 파일 끌어다 놓기

3.7 보조 도구

- **박스영상 만들기.bat** — OpenCV 가 있는 파이썬을 찾아 `draw_boxes.py` 를 돌린다. 없으면 "START.bat 을 먼저 한 번 실행하라"고 안내한다.
- **exe 다시 만들기.bat** — Pillow 가 있으면 `make_icon.py` 로 아이콘을 새로 그리고, 윈도우 기본 C# 컴파일러로 exe 를 다시 빌드한다. 컴파일러가 없으면 "START.bat 을 쓰면 결과는 같다"고 알려 준다.
- **make_icon.py** — 아이콘을 코드로 그린다. 1024px 로 크게 그린 뒤 256px 로 줄여 계단현상을 없애고, 16~256px 여섯 크기를 한 .ico 에 담는다. 그림 내용은 잔디 줄무늬 · 하프라인 · 센터서클 · 공 · 추적 상자 모서리다.

4 장

app.py — 사람이 만지는 부분

4.1 구조

tkinter 창 하나에 622행. 중요한 설계는 **분석을 별도 스레드에서 돌리고, 스레드는 화면을 직접 건드리지 않는다는** 것이다. tkinter 는 다른 스레드에서 위젯을 만지면 불안정해지기 때문이다.

분석 스레드 → `msg_q` (큐에 메시지만 넣는다) → 메인 스레드가 100ms 마다 꺼내서 화면 갱신
메시지 종류는 넷 — `log` (로그 한 줄) · `progress` (현재/전체/남은시간) · `preset` (GPU 확인 결과에 따른 프리셋 제안) · `done` (끝났음).

4.2 프리셋

프리셋	모델	해상도	프레임간격	설명
빠르게 (GPU 없을 때)	축구 전용	640	3	GPU 가 없으면 자동 선택된다
보통	축구 전용	960	2	640 대비 약 1.8배 느림
정확하게 (GPU 필요)	축구 전용	1280	1	전 프레임
직접 설정	—			아래 값을 손으로 조절. 기본 선택값

프리셋이 전부 축구 모델을 가리키게 된 이유

예전에는 프리셋이 전부 COCO 모델을 가리켰다. GPU 가 없으면 '빠르게'가 자동 적용되므로, `config.json` 에 축구 모델을 넣어놔도 **앱을 켤 때마다 조용히 yolo11n 으로 덮어써졌다.**

같은 720p 영상 실측 — COCO yolo11n @640 은 프레임당 **12.1명**(신뢰도 0.21), 축구 모델 @640 은 **23.1명**(신뢰도 0.82). 두 배 차이다.

같은 버그가 CLI 에도 남아 있었고(`best_local_model()` 주석), 지금은 양쪽 다 고쳐져 있다.

4.3 폴더 감시 — 복사 중인 파일을 건드리지 않는 법

`videos/` 를 2초마다 확인하지만 파일이 보이자마자 잡지 않는다. 파일 크기를 기억해 두었다가 **직전 확인 때와 크기가 같을 때만** 대기열에 넣는다. 복사가 끝나지 않은 파일을 분석하다 깨지는 것을 막는다. 지워진 파일의 기록은 정리해서, 오래 켜 뒤도 메모리가 쌓이지 않는다.

4.4 GPU 확인을 뒤에서 하는 이유

`import torch` 만으로 몇 초가 걸린다. 창이 뜨기 전에 이걸 하면 프로그램이 멈춘 것처럼 보이므로, 별도 스레드에서 확인하고 결과를 큐로 보낸다. 그리고 **사용자가 프리셋을 이미 만졌으면 덮어쓰지 않는다** ('`직접 설정`' 상태일 때만 자동 적용).

4.5 pythonw.exe 대비

`pythonw.exe` 로 띄우면 `sys.stdout` 과 `sys.stderr` 이 `None` 이다. 그 상태에서 `sys.stdout.write` 를 직접 부르는 라이브러리(`tqdm` 등)가 터진다. 그래서 시작하자마자 둘을 `logs/console.log` 로 돌려놓는다. 창 생성 자체가 실패하는 경우에 대비해 트레이스백을 `logs/crash.log` 에 남기고 메시지 상자로 알린다 — 콘솔이 없으면 오류를 볼 방법이 아예 없기 때문이다.

5 장

tracker.py (1) — 분석을 시작하기 전에

2,593행짜리 이 파일의 구조는 의외로 단순하다.

준비(5장) → 프레임 루프(6장) → 후처리(7장) → 저장

어려운 것은 그 사이에 낀 필터들인데, 필터 하나하나가 "무엇을 잘못 잡았는가"라는 구체적 관찰에서 출발한다. 코드 주석에 그 관찰과 실측 수치가 거의 다 적혀 있다.

5.1 Config — 80개 설정값

@dataclass 하나에 모든 기준값이 들어 있고 config.json 으로 저장·복원된다. 읽을 때 기본값의 타입에 맞춰 변환하고, 안 맞으면 그 항목만 기본값을 지킨다.

```
current = getattr(cfg, key)
if isinstance(current, bool): value = bool(value)
elif isinstance(current, int): value = int(value)
...
except (TypeError, ValueError): continue    # 이 항목만 기본값 유지
```

손으로 잘못 고친 config.json 하나 때문에 분석 한참 뒤 엉뚱한 지점에서 터지는 것을 막는 장치다. 모르는 키는 그냥 무시하므로, 옛 설정 파일도 그대로 읽힌다.

5.2 모델 준비 — GPU 가 없으면 OpenVINO 로 바꿔 둔다

```
if not cfg.use_openvino or has_cuda():
    return YOLO(cfg.model)          # GPU 가 있으면 원본 .pt 가 더 빠르다
...
cache = root / "models" / f"{stem}_{cfg.imgsz}_openvino_model"
if not (cache / "metadata.yaml").exists():
    exported = YOLO(cfg.model).export(format="openvino", imgsz=cfg.imgsz)
```

CPU 추론이 1.5~2배 빨라진다(30분 영상 기준 20.3분 → 12.8분). 변환본은 해상도마다 다르므로 해상도를 파일명에 붙여 캐시한다. 변환에 실패해도 원본으로 되돌아가 계속 진행한다.

지금 설정에서는 이 가속이 꺼져 있다

config.json 의 use_openvino 가 false 다. GPU 가 없는 PC 라면 이것만 켜도 분석 시간이 3분의 2로 줄어든다. 처음 한 번 1~3분의 변환 시간이 든다.

5.3 트래커 설정 자동 생성 — 두 가지를 바로잡는다 중요

resolve_tracker() 는 ultralytics 의 기본 bytetrack.yaml 을 읽어 값을 고친 사본을 만들어 쓴다. 고치는 것은 두 가지다.

(1) 신뢰도 임계값을 conf 에 맞춘다

```
settings["track_high_thresh"] = cfg.conf          # 0.10
settings["new_track_thresh"]  = cfg.conf          # 0.10 (기본 0.25)
settings["track_low_thresh"]  = max(0.01, cfg.conf * 0.5)
```

ByteTrack 은 `new_track_thresh` (기본 0.25) 미만인 검출로는 **새 트랙을 시작하지 않는다**. 사용자가 `conf=0.1` 로 낮춰 약한 검출까지 받기로 했는데 트래커 임계값이 0.25 로 남아 있으면, 그 검출들은 표에 잡히되 **track_id** 가 -1 로 찍힌다. 이 기능이 그것을 막는다.

(2) track_buffer 를 stride 로 나눈다

```
stride = max(1, int(cfg.vid_stride))              # 3
buffer_frames = max(5, round(int(cfg.track_buffer) / stride)) # 30 / 3 = 10
settings["track_buffer"] = buffer_frames
```

1판 정정 ①

1판에서 저는 "track_buffer 30 은 원본 프레임 기준이라 stride 3 이면 실질 10프레임뿐"이라고 썼는데, **이는 코드를 잘못 읽은 것이다**. 사실은 그 반대다 — 코드가 **일부러** stride 로 나눠서 **원본 프레임 기준의 의미를 유지**한다. 트래커는 3장마다 한 장만 보므로, 30 을 그대로 두면 원본 기준 90프레임(3.6초)을 기억하는 셈이 되어 **그 사이 다른 선수가 그 자리에 들어와 ID 를 물려받기 딱 좋다**. 나눠서 10으로 두면 원본 기준 30프레임(약 1.2초)이 된다.

결론은 바뀌지 않는다 — 1.2초는 여전히 짧으므로 `track_buffer` 를 올리는 것은 유효한 개선이다(13장). 다만 "버그라서 올려야 한다"가 아니라 "의도된 값이 이 영상에는 짧다"가 맞다.

5.4 팀 색 보정 — 유니폼 색으로 두 팀 가르기

심판은 경기장 안에서 뛰므로 위치로는 못 거른다. 그래서 색을 쓴다. 분석 시작 전에 한 번만 한다.

단계	함수	내용
①	<code>torso_color()</code>	상자에서 위 15~50% · 가운데 절반만 잘라낸다(머리와 다리를 뺀다). 잔디 초록 픽셀을 뺀 뒤 남은 픽셀의 Lab 색공간 a·b 두 채널 중앙값을 그 사람의 색으로 삼는다. 밝기(L)를 버리는 것이 핵심 — 그들의 선수와 햇빛 아래 선수를 같은 색으로 묶기 위해서다.
②	<code>calibrate_teams()</code>	영상 전체에서 균등 간격으로 16장만 뽑아 상체 색을 모은다. 이때 잔디 밖 사람은 빼고, '선수' 클래스에서만 뽑는다(심판·골키퍼 유니폼이 섞이면 군집이 흐려진다). 12명 미만이면 팀 구분을 포기한다.
③	<code>_kmeans()</code>	직접 구현한 작은 k-means(sklearn 의존을 피하려고). 4덩어리를 만들고 가장 큰 두 덩어리를 양 팀으로 본다. 초기 중심은 k-means++ 비슷하게 서로 먼 점부터 고른다.
④	군집 병합	같은 팀이 그늘·조명 때문에 두 군집으로 쪼개지는 일이 잦다. 남은 군집이 어느 팀 색에 "두 팀 사이 거리의 0.5배" 보다 가까우면 그 팀으로 합친다. 어디에도 안 붙으면 -1.

초록 유니폼 문제 — 필터가 상체를 통째로 지우는 경우

①의 "잔디 초록을 뺀다"는 단계가 유니폼이 초록 계열일 때 상체를 전부 지워 버린다. 실측 — 브라질(노랑) 대 멕시코(초록) 영상에서 상체 36개를 재니 **36개 전부가 잔디로 판정**됐다. 초록 유니폼은 당연하고, 노란 유니폼도 색조가 34라 잔디 범위(30~95) 하한 안쪽에 들어갔다. 팀 보정 표본이 342명 중 41명으로 줄고 96%가 미분류됐다.

그래서 `torso_grass_fallback` 이 있다 — 남은 픽셀이 12개 미만이면 지우지 말고 패치 전체를 쓴다. 상체 패치는 원래 대부분이 유니폼이라 잔디가 조금 섞여도 중앙값이 버틴다. 같은 영상에서 표본이 41 → **342명**으로 돌아왔고 두 군집이 거리 40.2 로 잘 갈렸다.

team_imgsz — 16장만 크게 보는 트릭

검출과 팀 구분은 요구가 다르다. 축구 모델은 640 에서도 선수를 거의 다 찾지만(22.7명 vs 1280의 23.0명), 팀 색은 상체 픽셀이 있어야 뽑힌다. 640 에서는 상체가 20px 남짓이라 실측상 **41%가 team=-1 로 섰다**.

팀 색 보정은 영상 전체에서 16장만 보므로, **그 16장만 높은 해상도로 처리**하면 전체 비용은 거의 안 늘면서 팀 구분만 좋아진다. 지금 설정은 `team_imgsz: 960` (검출은 640) — 잘 잡혀 있다.

5.5 구간 분석과 워밍업

```
python tracker.py "경기.mp4" --from 12:00 --to 15:00
→ results/경기 12m00s-15m00s/ 에 따로 저장 (프레임 번호는 원본 기준)
```

90분 경기 전체는 CPU 로 몇 시간이 걸리므로 구간 분석이 현실적이다. 구간을 잘못 넣는 실수가 흔해서 **네 가지를 미리 검사**한다 — 음수 시각, 끝이 시작보다 앞, 시작이 영상 길이를 넘음, 끝이 영상 길이를 넘음(이건 잘라서 진행). 조용히 빈 결과를 만들지 않고 이유를 알려 준다.

워밍업(`range_warmup_sec: 4.0`): 구간 시작보다 4초 앞서부터 추적기를 미리 돌리되 기록은 하지 않는다.

ByteTrack 은 갓 시작하면 트랙이 확정되지 않아 처음 1.5초쯤이 흔들리기 때문이다. 워밍업을 거치면 그 뒤로는 전체 분석과 93% 일치한다고 주석에 적혀 있다.

6 장

tracker.py (2) — 프레임 하나가 CSV 한 줄이 되기까지

6.1 검출 — 모델이 무엇을 아는가

`ModelProfile` 클래스가 "지금 쓰는 모델이 무엇을 아는지"를 정리하고, 이후 모든 필터가 이 판단에 따라 켜지거나 꺼진다.

모델	클래스	따라오는 결과
축구 전용 지금 쓰는 것	player / goalkeeper / referee / ball	스태프·관중은 어느 클래스에도 안 잡히고, 심판·골키퍼를 모델이 직접 알려준다. 공 크기·밝기·모양 검사가 켜진다 (기준값을 이 모델로 실측했으므로).
COCO 일반 yolo11n/s/m/l/x	person(0), sports ball(32)	감독·심판·볼보이·관중이 전부 선수와 같은 것으로 잡힌다. 공 물리 검사가 전부 꺼진다.

6.2 사람 필터 — 경기장 밖과 화면에 박힌 그래픽

잔디 마스크 (pitch_mask)

프레임을 **1/4 로 줄여서** 계산한다(충분하고 훨씬 빠르다). 순서는 이렇다.

- HSV 색상 30~95 · 채도 30 이상을 초록으로 골라낸다
- 모폴로지 연산으로 잔구멍을 메우고 자잘한 초록 노이즈를 지운다
- 초록이 화면의 **15% 미만이면 포기**하고 `None` 을 돌려준다(실내·리플레이 화면)
- 연결 요소 중 **가장 큰 덩어리**만 남긴다
- `_fill_holes()` — 테두리를 덧대고 바깥에서 물을 채운 뒤 뒤집어 **안쪽 구멍만** 메운다. 선수가 잔디를 가려 생긴 구멍은 채워지고 터치라인 바깥은 바깥으로 남는다
- 가장자리를 16px 안쪽으로 좁힌다(`pitch_shrink`)

그리고 사람 상자의 **발밑 점**이 마스크 밖이면 버린다. 중심점이 아니라 발밑인 이유는 사람이 땅을 딛고 서 있기 때문이다. 공에는 적용하지 않는다 — 높이 뜬 공은 관중석을 배경으로 걸친다.

축구 모델에서도 이 필터를 켜 두는 이유 — 실측으로 뒤집힌 판단

처음에는 "축구 모델은 스탭프·관중을 안 잡는다"며 컸는데 틀렸다. 전술 영상 10:00~13:00 구간에서 2초마다 91장을 뽑아, 잔디 밖으로 판정된 사람 **51건을 전부 눈으로 확인**한 결과:

- '선수' 라벨 44건 — 파란 트랙 위 스탭프, 노란 조끼 안전요원, 사진기자, 코치
- '심판' 라벨 7건 — 펜스 옆 안전요원 4, **광고판 글자 3**

진짜 선수는 0건이었다. 선수는 스로인·코너킥 때도 흰 선 밖 **잔디 여백** 위에 서므로 '초록 잔디 위' 기준으로는 빠지지 않는다.

실패해도 해가 없는 쪽으로 실패한다 — 관중 색이 잔디와 겹치면 마스크가 화면 전체를 잔디로 봐서 아무것도 안 거른다.

중계 그래픽 필터 (static_tracks) — 축구 모델에서는 꺼진다

```
static_on = cfg.drop_static_tracks and not profile.is_soccer
```

1판 정정 ②

1판에서는 이 필터가 항상 동작하는 것처럼 설명했는데, **축구 전용 모델을 쓰면 자동으로 꺼진다**. config.json 에 drop_static_tracks: true 라고 적혀 있어도 그렇다. 이유는 두 가지다 — 축구 모델은 중계 그래픽 속 인물을 애초에 잘 안 잡고, **불박이 판정이 골키퍼처럼 한자리에 오래 서 있는 사람을 지울 수 있기 때문이다**. 로그에도 "중계그래픽 필터는 이 모델에 불필요하므로 끕니다"라고 찍힌다.

판정 규칙 자체는 이렇다(COCO 모델을 쓸 때만 적용).

(이동 폭 ÷ 평균 키) < 0.10 그리고 등장 ≥ 15프레임 → 사람이 아니라 그래픽

이동 폭을 **키로 나누는** 이유: 멀리 있어 작게 잡힌 선수도 같은 자로 재기 위해서다. 픽셀 거리를 그대로 쓰면 먼 곳의 진짜 선수가 '안 움직인 것'으로 걸린다.

6.3 팀 판정 — 프레임 단위로는 표만 모은다

```
if int(cid) in profile.non_team:                # 모델이 심판·골키퍼라고 말해 준다
    team = -1
elif int(cid) in profile.team_targets and calib is not None:
    col = torso_color(frame_img, (x1,y1,x2,y2), cfg)
    team = assign_team(col, calib)              # 가장 가까운 군집
    team_votes[tid][team] += 1                  # ← 표만 모은다. 확정은 나중에
```

프레임마다 확정하지 않는 이유는 7.1 에서 다룬다. 요컨대 겹침·모션블러 한 번이면 한 선수의 팀이 0→-1→1 로 깜빡이기 때문이다.

6.4 공 파이프라인 — 여섯 단계

이 프로그램에서 가장 공들인 부분이다. 이유는 분명하다. **공 위치가 틀리면 기여도가 통째로 틀린다**. 공이 어디 있느냐로 "누가 소유자인가", "어느 팀이 공격 중인가", "위험한 공간이 어디인가"가 전부 결정된다.

왜 이렇게까지 하는가 — 실측 오검출

리버풀-레알 경기 50초 구간을 눈으로 전수 판정한 결과, 공 검출 **587건 중 281건(48%)이 페널티 마크**였다. 게다가 마크가 잡히는 프레임에서는 진짜 공을 대신 놓치는 일이 많았다. 무작위 104건 판정에서는 **흰 축구화가 19건(18%)**으로 가장 큰 오류였다.

① 공 전용 모델

선수 모델은 공을 잘 못 찾는다. 공은 지름 12px 남짓이라 선수와 함께 학습된 모델에서는 늘 뒷전이다. 같은 60프레임 실측 — 선수 모델 @1280 은 42장에서 발견하고 968ms/장, 공 전용 모델 @1280 은 50장에서 발견하고 134ms/장. **더 잘 찾으면서 7배 빠르다**. 공 전용 모델을 쓰면 주 모델의 공 검출은 통째로 버린다(같은 공을 두 줄로 적지 않기 위해).

해상도는 `ball_imgsz: 1920` (원본 해상도)이다. 1280 → 1920 으로 올리자 먼 쪽 터치라인의 공(7px 남짓)을 0.07 → **0.83** 신뢰도로 봤고, 공이 잡히는 프레임이 57.0% → 68.9% 가 됐다. 비용은 공 모델 140 → 330ms/장.

② 주시 확대 (ball_focus)

```
steps = min((src_frame - last_frame) / stride, 5.0) # 오래 놓쳤으면 멀리 밀지 않는다
px = last[0] + vel[0] * steps # 등속 예측 위치
→ 그 주변 480px 를 잘라 960 으로 키워 다시 검출
```

먼 공이나 선수 발밑의 공은 전체 화면에서 몇 픽셀이라 신뢰도가 바닥인데, 2배로 키우면 같은 공을 0.04 → 0.7~0.8 로 잡는다. 공을 놓친 지 2초가 지나면 그만 본다. 확대 창에서 찾은 것이 전체 화면에서 이미 본 공과 8px 안에 있으면 **신뢰도 높은 쪽만 남긴다**.

③ 네 가지 물리 검사

검사	기준	근거
크기 비율	0.16 ~ 0.36	공 박스 높이를 같은 프레임 사람 키의 중앙값 으로 나눈다. 픽셀 크기를 그대로 쓰면 '작다'와 '멀다'가 섞이는데, 마크 옆의 선수도 똑같이 멀어 작게 잡히므로 나누면 거리가 상쇄된다. 해상도마다 박스가 조이는 정도가 달라서 <code>ball_ratio_scale</code> (1920 은 0.89, 확대 창은 0.82)로 1280 기준으로 되돌린 뒤 검사한다. 실측 분포 — 진짜 공 0.170~0.319(중앙 0.208), 페널티 마크 0.134~0.232(중앙 0.159).
밝기	최대 ≥ 215	공은 포화된 흰색, 페인트 자국은 바랜 회백색. 실측 회색조 최대값 — 진짜 공 248 , 페널티 마크 196 , 사진기자석 216. 주변 잔디 대비(상대값)로도 재봤지만 더 나빴다 — 마크도 잔디보다는 밝기 때문이다. 절대값이 맞다 .

모양(채움비)	≥ 0.65	흰 덩어리 넓이 ÷ 그 덩어리의 외접 사각형 넓이. 공은 원이라 이론값 $\pi/4 \approx 0.785$, 실측 중앙값 0.77. 축구화는 끈·스트라이프 때문에 들쭉날쭉해 0.52. 중형비로도 재봤지만 못 쓴다 — 빠른 공은 모션블러로 가로로 늘어나 축구화와 구별이 안 된다. 채움 비율은 블러에도 버틴다.
정지	$\text{conf} < 0.30$	0.24~0.96초 전에도 8px 안에 후보가 있었으면 '멈춘 것'으로 본다. 약하면서 멈춰 있으면 뻘다 — 카메라맨·관중석·서 있는 선수의 흰 축구화. 이 규칙으로 빠진 75프레임을 전수 판정하니 가짜 59 · 진짜 16 이었고, 문턱을 0.4~0.5 로 올려도 가짜는 더 안 빠지고 진짜만 더 잃었다.

밝기 면제 규칙이 따로 붙어 있다. 날아가는 공은 모션블러로 흰색이 번져 밝기가 190~210 으로 떨어지기 때문이다. 조건이 셋이다 — 신뢰도 0.70 이상(정답 라벨의 마크는 최대 0.67), 밝기 180 이상(터치라인 밖 여백에 놓인 공 모양 물체가 146~163 이었다), 그리고 **움직이는 후보일 것**. 멈춘 공은 번지지 않으므로 밝게 찍히고 (골킥 대기 공 235~249), 페널티 마크는 제자리에 있으면서 밝기 181~208 이라 밝기만으로는 안 걸렸다. 이 '움직임' 조건으로 확대 결과의 마크가 118프레임 → 9프레임이 됐다.

④ 후보 하나 고르기 (BallPicker)

$$\text{점수} = \text{신뢰도} + 0.6 \times \exp(-\text{예측위치까지 거리} \div 140\text{px})$$

근거는 "경기장에 공은 하나뿐". 페널티 마크는 늘 제자리라 예측에서 벗어나고, 진짜 공은 매끄럽게 움직여 예측에 맞는다. 실측: 프레임간 이동폭 90% 지점이 102px → 69px.

15프레임 넘게 끊기면 이어붙일 근거가 없다고 보고 신뢰도만으로 고르며 **새 track_id** 를 준다.

스스로 강화되는 함정과 그 해법

연속성 보너스(최대 0.6)가 신뢰도 차이보다 크면 문제가 생긴다. 한번 페널티 마크에 물리면 마크는 늘 제자리라 보너스를 통째로 받고, 멀리 튄 진짜 공은 보너스를 못 받아 **영영 빠져나오지 못한다**.

실측 f342 — 진짜 공 $0.79+0.00=0.79$ / 페널티 마크 $0.54+0.56=1.10$ ← 이것이 뺏혔다.

해법은 "훨씬 확신하는 후보가 있으면 갈아탄다" (ball_switch_margin 0.20). 선택이 갈린 19개 프레임을 전수 확인하니 **19개 모두 갈아탄 쪽이 진짜 공**이었다. 갈아탄 직후에는 **속도를 0으로 초기화**한다 — 옛 자리에서 새 자리까지의 거리를 속도로 삼으면 다음 예측이 엉뚱한 곳을 가리켜 다시 마크로 되돌아가기 때문이다.

⑤~⑥ 궤적 잇기와 보간 — 7장에서 다룬다

이 둘은 프레임 루프가 아니라 **분석이 다 끝난 뒤 CSV** 를 다시 훑으며 한다. 전체 궤적을 봐야 판단할 수 있기 때문이다.

6.5 기록 — CSV 한 줄

```
frame, time_sec, track_id, class_id, class_name, conf,
x1, y1, x2, y2,      ← 박스 좌상단 / 우하단 (픽셀)
cx, cy,              ← 박스 중심
fx, fy,              ← 발밑 중심점 = ((x1+x2)/2, y2)  "지면 좌표 변환용"
w, h, team, interpolated
```

공 행은 f_x, f_y 에 (c_x, y_2) 를 적는다 — 공은 지면에 붙어 있지 않으므로 아래쪽 경계를 지면 접점으로 본다. team 은 빈칸이다.

진행 상황은 10프레임마다 GUI 로 보내고, **300프레임마다 파일을 flush** 한다 — 도중에 끊겨도 거기까지는 남는다.

7 장

tracker.py (3) — 분석이 끝난 뒤의 후처리

프레임 하나만 보서는 판단할 수 없는 것들이 있다. "이 트랙은 전체적으로 어느 팀인가", "이 트랙은 처음부터 끝까지 안 움직였나", "이 약한 공 후보가 확실한 공과 이어지나". 전부 **궤적 전체**를 봐야 알 수 있으므로 CSV 를 다 쓴 뒤에 한번 더 훑는다.

후처리 순서 — 이 순서가 중요하다

- ① `finalize_tracks()` : 팀 다수결 + 골키퍼 트랙 확정 + 불박이 사람/공 제거 + 심판 제외
- ② `chain_ball()` : 공 궤적 잇기 (확실한 공에서 앞뒤로 약한 후보를 따라간다)
- ③ `interpolate_ball()` : 남은 짧은 공백을 직선으로 메움
- ④ `summary.json` 저장 → ⑤ 결과 영상 만들기

②가 ③보다 먼저인 이유: 잇기로 실제 관측을 먼저 채워 넣어야, 보간이 메울 공백이 줄고 보간 구간도 짧아진다. ④가 ⑤보다 먼저인 이유는 7.6 참고.

7.1 팀 다수결 (`finalize_tracks`)

```
final = {tid: _decide_team(votes) for tid, votes in team_votes.items()}

def _decide_team(votes):
    return min(votes.items(), key=lambda kv: (-kv[1], kv[0] < 0, kv[0]))[0]
```

표가 가장 많은 팀으로 정하되, **동률이면 실제 팀(0/1)을 -1 보다 우선한다**. 멀어서 상체가 안 잡힌 프레임이 -1 로 세어지기 때문에, 동률에서 -1 을 고르면 손해다.

`drop_non_team` (심판 제외)도 이 단계에서 처리한다. 프레임 단위로 미리 버리면 **다수결에서 이겼을 선수가 중간중간 사라지므로**, 확정된 뒤에 버려야 맞다.

작업은 CSV 를 한 줄씩 흘려 읽어 임시 파일에 쓴 뒤 갈아끼우는 방식이다. 도중에 실패하면 원본을 그대로 남기고 `None` 을 돌려준다 — 몇 시간짜리 분석 결과를 날릴 수는 없다.

7.2 골키퍼 트랙 확정 — 모델이 골대 기둥을 골키퍼라고 부른다

가장 흥미로운 필터다. 실측이 이렇다.

- 진짜 골키퍼 트랙은 **골키퍼 라벨이 3~39%뿐**이고 나머지 프레임은 '선수' 라벨이었다.
- 반대로 **골대(세로 기둥·그물)는 골키퍼 라벨이 69~100%**였다. 학습 데이터에서 골키퍼가 늘 골대 옆에 서 있어서 모델이 "기둥 = 골키퍼"로 배운 것이다.

영상에서는 이것이 "번호가 났다 사라졌다 바뀌는" 현상으로 보였다 — 골키퍼일 때만 GK 로, 나머지는 번호 박스로 찍혔기 때문이다. 그래서 트랙 단위로 가른다.

조건	판정	처리
----	----	----

유니폼 색이 팀 0 또는 1	필드 선수	골대 근처에서 잠깐 잘못 찍힌 것. 골키퍼 라벨을 선수로 되돌린다.
팀 -1 이고 트랙 전체 신뢰도 중앙값 < 0.50	골대	선수 라벨 행까지 트랙을 통째로 뺀다.
그 외	골키퍼	트랙 전체를 골키퍼로 통일한다.

임계값 0.50 의 근거: 트랙 전체 신뢰도 중앙값이 골대는 0.22·0.30·0.34·0.34·0.36·0.41, 골키퍼는 0.60·0.65·0.75·0.77·0.80·0.80·0.81 이었다. 골키퍼 라벨이 붙은 행만 보면 0.41 / 0.45 로 거의 붙어 있는데, 진짜 골키퍼는 **선수로 찍힌 행**의 신뢰도가 높아 트랙 전체로 보면 0.41 / 0.60 으로 벌어진다. 신뢰도는 20칸 히스토그램으로 모아 두었다가 중앙값을 낸다.

7.3 불박이 공 제거 (static_ball_tracks)

공은 경기장에서 가장 많이 움직이는 물체다. 프리킥으로 멈춰 서 있어도 결국 차이므로 **트랙 전체로 보면 반드시 움직인다**. 실측에서 흰 옷 입은 선수의 몸통이 12프레임 연속(움직임 1px) 공으로 잡혔는데, 그런 것을 뺀다.

(이동 폭 ÷ 평균 공 크기) < 1.0 그리고 등장 ≥ 5프레임 → 공이 아니다

7.4 궤적 잇기 (chain_ball) — 시간축으로 진짜 공을 가려낸다

한 프레임만 보면 달리는 선수의 흰 축구화는 공과 구별되지 않는다 — 밝기 222~250, 채움 0.62~0.83, 크기비 0.20~0.27 로 **전부 공의 범위** 안에 있다. 차이는 **시간축**에 있다. 진짜 공의 약한 후보들은 서로 이어져 확실한 공으로 연결되지만, 축구화는 어떤 확실한 공과도 이어지지 않는다.

끝점(anchor) 고르기 — 세 조건을 다 만족해야 한다

1. 그 프레임에 뿔힌 공의 신뢰도가 **0.60 이상**
2. **전체 화면**에서도 같은 자리(6px 안)를 0.60 이상으로 봤을 것 — 확대 창은 축구화도 0.7 로 부풀리므로, 확대 창 신뢰도로 끝점을 잡으면 **축구화 궤적이 끝점이 되어 버린다**
3. 앞뒤 1~2 처리프레임에 40~80px 안으로 이어지는 공이 있을 것

끝점에서 앞뒤로 따라가기

```
tol = 28.0 + 0.6 × |속도| × (놓친횟수+1)      # 빠를수록 넉넉하게
후보 선택 = min(거리 - 20 × 신뢰도)          # 가깝고 확신 있는 것
속도 갱신 = 0.5 × 이전속도 + 0.5 × 실제이동  # 급변을 눌러 준다
연속 2회까지는 후보가 없어도 예측 위치로 계속 간다
```

이렇게 이어진 길이가 **3 이상**이고 기존 선택과 **30px 이상** 떨어져 있을 때만 바꾼다. 실측(전술 10:00~13:00) 오프라인 재현에서 교체 84건 · 추가 161건이었고, 교체 78건을 눈으로 비교하니 좋아진 것 약 50, 나빠진 것 약 5(점선 조각·손), 불분명 약 20 이었다. 추가 64건 중 틀린 것은 약 4건.

허용 오차 28px 의 근거: 14px 는 속도를 모를 때 너무 좁아 이어지지 않았고, 28px 에서 문제 구간(10:06, 공이 뜬 순간 놓치자 확대 창이 날아가던 방향의 축구화를 0.3~0.7 로 잡았던 구간)이 전부 바로잡혔다.

7.5 짧은 공백 메우기 (interpolate_ball)

양 끝이 **엄격한 끝점**이고, 공백이 **0.4초 이하**이고, 두 끝점 사이 속도가 **초당 선수 키 20배 이하**일 때만 직선으로 메운다. 메운 행은 `interpolated=1`, `conf=0` 으로 표시한다.

0.4초라는 상한이 나온 과정 — 처음엔 2.0초로 넣었다가 틀렸다

측정된 공을 일부러 지우고 양옆 끝점으로 보간해 실제 위치와 비교했다(단위: 선수 키).

빈 구간	오차 중앙값	키의 1/3 이내 비율
0.16초	0.01	100%
0.6초	0.18	76%
0.8초	0.28	58%
1.6초	0.91	17%

2.0초로 메운 점 52개를 눈으로 보니 제대로 맞은 것이 **29%뿐**이었다. 공은 짧은 시간에도 방향을 바꾸고(드리블·패스) 카메라가 패닝해서 픽셀상 경로가 직선이 아니다. 상한별 이득은 0.3초 +3%p / 0.4초 +4%p / 0.5초 +5%p 인데, 0.4초면 오차 90% 지점에서도 선수 키의 8%(약 5px)라 공 위에 떨어진다.

롱패스처럼 몇 초짜리 공백은 이 방법으로 메울 수 없다. 선을 그으면 틀린다.

7.6 저장 순서 — 몇 시간짜리 분석을 지키는 설계

1. tracks.csv 완성
2. 후처리 (팀 · 골키퍼 · 볼박이 · 잇기 · 보간)
3. summary.json 저장 ← 여기서 '이 영상은 끝났다'가 확정된다
4. 결과 영상 만들기 (수십 분)
5. 영상이 성공하면 summary.json 을 한 번 더 갱신 (result_video, elapsed_sec)

3과 4의 순서가 핵심이다. 긴 경기 영상은 인코딩에만 수십 분이 걸리는데, 그 도중에 끊겨서 summary.json 이 없으면 **'이미 한 건 건너뛰기'가 안 먹어 몇 시간짜리 분석을 처음부터 다시 돌게 된다.** 이 순서면 영상이 실패하거나 중단돼도 분석 결과는 남는다. (그래도 사고가 나면 `recover_summary.py` 가 CSV 에서 되살린다.)

한 장도 못 읽었으면 CSV 와 폴더를 지우고 `None` 을 돌려준다 — summary.json 을 안 써야 다음에 재시도되기 때문이다.

7.7 결과 영상은 왜 draw_boxes.py 를 부르나

ultralitics 의 `save=True` 를 안 쓰는 이유가 있다. 그쪽은 **처리한 프레임만** 원본 fps 로 이어붙이기 때문에, `vid_stride` 를 쓰면 3배속으로 똑똑 끊기는 영상이 나온다. `draw_boxes.py` 는 원본을 처음부터 다시 훑으면서 분석 안 한 프레임에는 직전 박스를 남겨두므로 **길이도 속도도 원본과 같고**, 팀 색까지 반영된다.

구간 분석이면 영상도 그 구간만 그린다. 예전에는 구간을 안 넘겨서 5초만 분석해도 90분 경기 전체를 인코딩했다 (10분 만에 548MB).

8 장

내 config.json 으로는 실제로 무엇이 켜져 있나

이 장이 1판에는 없던 부분이다. 코드에는 "이 조건이면 이 필터가 꺼진다" 같은 분기가 많아서, **설정**에 `true` 라고 적혀 있어도 실제로는 안 되는 것들이 있다. 지금 설정으로 실제 무엇이 도는지 하나씩 대조한다.

8.1 설정 대조표

설정	내 값	실제	결과
<code>model</code>	soccer_yolo11m.pt	축구 모델	선수·골키퍼·심판·공을 직접 구분. 공 물리 검사 3종이 켜진다.
<code>ball_model</code>	soccer_ball_yolo11n.pt	켜짐	공을 별도 모델로 찾고, 공 track_id 가 1,000,000번대가 된다. 기여도 코드가 공 행을 인식하는 유일한 통로(11.4).
<code>imgsz</code>	640	적용	축구 모델은 640 에서도 선수를 거의 다 찾는다. 다만 박스가 작아 추적이 끊기기 쉽다.
<code>vid_stride</code>	3	위험	3프레임에 1장. 처리 간격 0.125초. ID 파편화의 최대 원인(12장). README 자신이 "선수 번호가 자주 바뀌면 '보통'(=2)을 쓰라"고 권한다.
<code>conf</code>	0.10	적용	약한 검출까지 받고 뒤에서 물리로 거르는 전략. <code>auto_tracker_thresh</code> 가 켜져 있어 트래커 임계값도 0.25 → 0.10 으로 함께 내려간다.
<code>match_thresh</code>	0.9	적용	이미 최대치로 올려둔 상태. 실측 0.8→0.9 로 ID 233→139개.
<code>track_buffer</code>	30	10으로 환산	stride 3 이므로 처리 프레임 기준 10장 = 원본 기준 30프레임(약 1.2초). 의도된 동작이다(5.3).
<code>team_imgsz</code>	960	좋음	팀 색만 960 으로 뽑는다. 16장만 해당되어 비용은 거의 안 든다.
<code>pitch_filter</code>	true	동작	축구 모델에서도 켜 둔다(6.2 의 51/51 실측).
<code>drop_static_tracks</code>	true	안 뚫	축구 모델이면 자동으로 꺼진다. 로그에 "중계그래픽 필터는 이 모델에 불필요하므로 끕니다"가 찍힌다.
<code>drop_non_team</code>	false	주의	심판·골키퍼가 <code>team=-1</code> 로 CSV 에 남는다. 기여도 코드는 이것을 세 번째 팀으로 세어 중단한다(11.3).
<code>use_openvino</code>	false	기회 손실	GPU 가 없다면 켜는 것만으로 분석 시간이 약 2/3가 된다(30분 영상 20.3분 → 12.8분).

skip_done	false	—	이미 분석한 영상도 다시 돌린다. 실험 중이라면 맞는 설정이다.
save_video	true	동작	확인용 영상을 만든다. 눈으로 검증할 수단이므로 커두는 것이 맞다.
ball_focus	true · 480→960	동작	먼 공을 잡는 핵심 장치. 다만 전체 처리 시간의 상당부분 을 차지한다.
ball_imgsz	1920	동작	공 발견률을 크게 올린다(57% → 69%). ball_ratio_scale: 0.89 와 짝이다 — 한쪽만 바꾸면 크기 검사가 통째로 어긋난다.

8.2 그래서 지금 실제로 도는 파이프라인

선수 : 축구모델@640 검출 → 잔디 마스크(발밀) → 상체색@960 으로 팀 표 수집 → ByteTrack(conf 0.10, match 0.9, buffer 10)

→ **중계그래픽 필터 없음** → 팀 다수결 → 골키퍼 트랙 확정 → **심판 제거 안 함**

공 : 공모델@1920 + 확대창 480→960 → 크기(0.16~0.36) · 밝기(215) · 모양(0.65) · 정지 검사
→ 후보 1개 선택(연속성+0.6, 갈아타기 0.20) → 불박이 공 제거 → 궤적 잇기 → 0.4초 보관

8.3 이 설정의 약점 세 가지

1. **vid_stride 3 + imgsz 640** — 둘이 겹쳐 최악의 조합이다. 박스가 작는데(640) 시간 간격까지 넓어서(0.125초) 박스 겹침이 0 이 되기 쉽다. **둘 중 하나만 바뀌어도 크게 개선된다.**
2. **OpenVINO 꺼짐** — GPU 가 없다면 아무 손해 없이 시간을 1/3 줄일 수 있다. stride 를 3 → 2 로 올리면서 OpenVINO 를 켜면 **총 시간은 거의 그대로 두고 추적 품질만 올릴 수 있다.**
3. **심판이 CSV 에 남음** — 기여도 단계에서 반드시 걸러야 한다. 다만 축구 모델은 골키퍼도 -1 로 표시할 수 있으므로, **class_name** 으로 골키퍼는 남기고 **referee** 만 **빼야** 한다. 설정에서 **drop_non_team: true** 로 바꾸면 골키퍼까지 사라질 수 있어 위험하다.

8.4 1판에서 잘못 쓴 것 두 가지 (정정)

정정 ① · track_buffer

1판: "기본 30은 원본 프레임 기준이라 stride 3이면 실질 10프레임뿐이다"(버그처럼 서술)

사실: resolve_tracker() 가 **일부러** stride 로 나눠 원본 기준 의미를 보존한다. 나누지 않으면 오히려 3.6초를 기억해 다른 선수에게 ID 가 넘어간다.

영향: 권고 자체(값을 올려 보라)는 유효하지만, 이유가 다르다.

정정 ② · 중계 그래픽 필터

1판: 이 필터가 항상 동작하는 것으로 설명

사실: `static_on = drop_static_tracks and not is_soccer` — 축구 모델에서는 자동으로 꺼진다. 불박이 판정이 골키퍼처럼 한자리에 오래 서 있는 사람을 지울 위험 때문이다.

영향: 지금 설정에서는 이 필터를 신경 쓸 필요가 없다.

9 장

결과물 읽는 법

9.1 tracks.csv

열	값	뜻	기여도에서의 쓰임
frame	정수	원본 영상 기준 프레임 번호(구간 분석이어도 원본 기준)	필수 — 시간축
time_sec	초	영상 시작 기준 경과 시간	fps 추정, 표본검사 시각
track_id	정수 공은 1,000,000~	같은 선수는 같은 번호. 놓치면 -1	필수 — 선수별 집계 기준이자 공 행 판별 통로
class_id class_name	player / goalkeeper referee / ball	무엇으로 인식했는지	심판 제외·골키퍼 지정에 꼭 필요(현재 열 이름이 안 맞아 미인식 → 11.2)
conf	0~1	신뢰도. 보간된 공은 0	공 신뢰도 하한 필터에 쓸 수 있다
x1 y1 x2 y2	픽셀	박스 좌상단·우하단	직접 안 씀(draw_boxes 용)
cx cy	픽셀	박스 중심	공 위치로 적합(공은 지면에 안 붙어 있다)
fx fy	픽셀	발밀 중심점 = $((x1+x2)/2, y2)$	가장 중요 — 호모그래피를 적용해 X, Y(미터)로 만들 좌표
w h	픽셀	박스 크기	공 크기 검사의 기준자(h 중앙값)
team	0 / 1 / -1 / 빈칸	양 팀 / 팀 아님 / 공	필수 — 공수 구분(규약 변환 필요 → 11.3)
interpolated	0 / 1	1이면 측정이 아니라 메운 추정 위치	현재 무시됨 → 11.5

9.2 summary.json — 품질 점검표

분석이 잘 됐는지 판단하는 데 가장 먼저 볼 파일이다. 특히 굵게 표시한 셋을 본다.

필드	보는 법
unique_track_ids	가장 중요. 22명 경기에서 20초 구간이면 30 안쪽이 정상, 100을 넘으면 파편화 상태. 이 값 하나로 '선수별 기여도를 믿어도 되는지'가 갈린다.
ball_frames / frames	공 발견률. 전술 카메라 1080p 기준 75~80% 가 제작자가 밝힌 정상 범위다. 50% 아래면 원인을 찾아야 한다.
class_counts	player / goalkeeper / referee / ball 각각 몇 건인지. 골키퍼가 0 이면 골키퍼 판정이 실패한 것이고, 기여도 단계에서 SC 가 부풀려진다.

<code>dropped_outside_pitch</code>	잔디 밖이라 뺀 사람 수. 중계 영상일수록 크다.
<code>ball_rejected_by_size /</code> <code>ball_rejected_dim /</code> <code>ball_rejected_ragged</code>	공 후보를 크기·밝기·모양으로 뺀 건수. <code>ball_rejected_dim</code> 이 전체 후보의 90%를 넘으면 영상이 어두운 것이므로 <code>ball_min_bright</code> 를 낮춰야 한다(로그에도 경고가 뜬다).
<code>ball_chain_replaced /</code> <code>_added</code>	궤적 잇기가 바꾼 행 / 새로 넣은 행
<code>ball_interpolated_rows</code>	보간으로 메운 행 수
<code>ball_focus_calls /</code> <code>ball_frames_from_focus</code>	확대 재검출을 몇 번 돌렸고 그중 몇 번이 실제로 공이 됐는지. 비율이 낮으면 <code>ball_focus</code> 를 꺼서 시간을 아낄 수 있다.
<code>processed_fps /</code> <code>elapsed_sec</code>	속도. 설정을 바꿨을 때 비용을 비교하는 기준.
<code>config</code>	그 분석에 쓴 설정 전체가 통째로 들어간다. 나중에 결과를 비교할 때 결정적이다.

9.3 확인용 영상

표시	뜻
빨강 / 파랑 박스 + 숫자	양 팀 선수, 숫자는 <code>track_id</code> . 이 숫자가 얼마나 자주 바뀌는지가 핵심 점검 항목
주황 박스 + GK	골키퍼
보라 박스 + REF	심판
노란 원 + 십자	공(추정값)
회색 점선 원	공(0.4초 이하 공백을 추정으로 메운 것)

이 도구가 왜 중요한가 — 평균은 거짓말을 한다

검출 오류는 **평균 뒤에 숨는다**. "프레임당 22.7명 검출"이라는 숫자는 그중 셋이 관중이어도 똑같이 나온다. 오검출을 실제로 찾아내는 방법은 통계가 아니라 **하나씩 잘라서 눈으로 보는 것**뿐이다. 이 프로그램의 필터 대부분(페널티 마크 48%, 축구화 18%, 골대 기둥, 잔디 밖 51/51)이 전부 그렇게 발견됐다. 기여도 코드에도 같은 목적의 '표본검사' 시트가 있다(10.8).

10 장

기여도 계산 — SC · PR · PA · DPI · TC

여기서부터는 `soccer_dpi_pa_v7.py` 의 이야기다. 앞의 16개 파일과는 별개 프로그램이고, `tracks.csv` 를 (변환해서) 받아 선수별 기여도 표를 낸다.

10.1 무엇을 재려는 것인가

골·어시스트·패스 성공률은 **공에 닿은 사람**만 기록한다. 그런데 축구에서 일어나는 일의 대부분은 공 없이 일어난다. 수비수가 공간을 미리 막아 패스 선택지를 지운 것, 미드필더가 소유자에게 붙어 시간을 빼앗은 것, 공격수가 좋은 자리에 서서 수비를 끌고 간 것 — 어떤 기록지에도 안 남는다. 그것을 세 축으로 나눠 잴다.

지표	무엇을	언제 나오나	한 줄 정의
SC	공간 통제	팀이 수비 할 때	위험한 공간을 상대보다 먼저 덮고 있었는가
PR	압박 기여	팀이 수비 할 때	공 가진 상대에게 얼마나 붙었고 얼마나 빠르게 쫓았는가
PA	패스 유인	팀이 공격 할 때	동료가 나에게 패스하고 싶을 만큼 좋은 선택지였는가
DPI	수비 합성	—	$0.5 \cdot z(SC) + 0.5 \cdot z(PR)$
TC	총 기여	—	셋을 순위 정규화해 1/3씩 합친 값 (+ 포지션 보정분 <code>TC_adj</code>)

SC·PR 과 PA 는 서로 다른 프레임에서 나온다

우리 팀이 공을 갖고 있는 동안 우리 선수에게는 PA 만 쌓이고 SC·PR 은 쌓이지 않는다. 그래서 **양 팀이 공수를 주고받는 구간**이 충분해야 세 지표가 다 모인다. 짧은 클립에서 한 팀이 계속 공을 갖고 있었다면 한쪽은 수비 기록이, 다른 쪽은 공격 기록이 아예 없어 TC 를 낼 수 없다.

10.2 계산 전 준비

순서	단계	내용
①	자체 검증 12항목	등속 5 m/s 가 정확히 5.00 으로 나오는지, Shapley 배분의 합이 총량과 일치하는지(효율성 공리), 위험도 가중치 방향이 맞는지 등. 하나라도 실패하면 거기서 멈춘다 — 틀린 계산으로 낸 표를 보여주지 않기 위해서다.
②	좌표계 판정	원점이 구석인지 중앙인지 자동 판정. 기준은 좌표의 중앙값 이다. 최소값으로 판정하면 스로인하러 라인 밖에 선 선수 한 명 때문에 판정이 뒤집혀 전체 좌표가 반 경기장만큼 밀린다.
③	경기장 밖 제거	경계에서 3m 넘게 밖에 있는 행을 버린다. 스로인·코너킥 선수를 위해 여유를 둔다.

④	공격 방향 판정	골키퍼는 자기 골대 앞에 상주하므로 팀별 최소 상주 X 를 비교해 어느 팀이 $x=0$ 골대를 지키는지 알아낸다. 틀리면 위험도 가중치가 상대 골대에 걸리고 PA 전진 부호가 뒤집혀 결과가 통째로 무의미해진다. 전·후반이 한 파일에 있으면 방향이 중간에 바뀌므로 그것도 검사해 경고한다.
⑤	ID 재연결	ID 기준 이동속도의 95% 지점이 15 m/s를 넘으면 헝가리안 매칭으로 다시 이어 붙인다. 사람이 갈 수 있는 거리를 넘는 연결은 거부한다. 12장에서 이 단계가 문제를 일으킨다.
⑥	결측 보간	선수는 0.3초, 공은 0.2초 이하 끊김만 선형보간. 메운 행은 표시해 둔다.
⑦	속도 계산	중앙차분(앞뒤 3프레임). 11 m/s(=39.6 km/h)를 넘으면 이상치로 버린다 — 프로 실측 최고속도가 34~37 km/h 이므로 그 위는 추적 오류다.

10.3 공 소유 판정 — 모든 계산의 기준점

1. **가장 가까운 선수**를 찾아 POSS_RADIUS (데이터를 보고 2~8m 중 자동 결정) 이내면 소유자로 인정
2. **데드볼 제외** — 공이 라인 밖 1m를 넘으면 소유자를 두지 않는다. 90분 경기에서 공이 실제로 살아 있는 시간은 55~60분뿐이고, 스로인을 기다리며 자리 잡는 배치를 공간 다툼으로 계산하면 안 된다
3. **공중볼 제외** — 공 높이가 1.5m를 넘으면 소유자를 두지 않는다. 헤더 경합은 점프와 타이밍으로 결정되므로 지면 2D 거리로는 판정할 수 없다. (단, tracks.csv에는 높이 열이 없어 현재는 적용되지 않는다)
4. **전환 인정** — 1.5초보다 짧으면서 **앞뒤가 같은 팀**인 구간은 잡음으로 보고 흡수한다. $A \rightarrow B \rightarrow A$ 의 짧은 B는 지우고, $A \rightarrow B \rightarrow C$ 의 B는 실제 전환이므로 남긴다
5. **usable 판정** — 소유팀이 정해졌고 **양 팀 모두 명목 인원의 80% 이상**이 추적된 프레임만 쓴다

usable 이 왜 필요한가

SC는 경기장 전체 격자를 "가장 가까운 수비수"에게 나눠 주는 계산이다. 뒤쪽 수비수가 화면 밖으로 빠지면 **남은 수비수가 그 빈 자리의 공간 점유까지 대신 받아** SC가 부풀려진다. 한 대의 카메라로 만든 좌표는 경기장의 2/3 정도만 담으므로, usable 비율이 낮게 나오는 것이 오히려 정직한 신호다.

10.4 SC — 공간 통제 (Shapley 배분)

1걸음 · 격자와 위험도

$$w(\text{칸}) = \exp(-\text{골대까지거리} \div 18) \times \exp(-\text{공까지거리} \div 20) \times \text{칸넓이}$$

1m 격자이므로 105×68 경기장이면 약 7,140칸. "위험하다"는 것은 **내 골대에 가깝고 동시에 공에 가깝다**는 뜻이다. 둘 중 하나만 만족하는 칸은 값이 급격히 떨어진다.

예 — 내 골대에서 20m, 공에서 15m 떨어진 칸: $\exp(-20/18) \times \exp(-15/20) = 0.329 \times 0.472 = \mathbf{0.156}$. 골문 바로 앞(골대 5m, 공 5m)이면 $0.757 \times 0.779 = \mathbf{0.590}$ 으로 약 네 배다.

2걸음 · 누가 그 칸을 통제하나

그 칸까지의 거리가 **가장 가까운 공격수보다 가까운** 수비수들이 그 칸을 통제한다고 본다. 한 명일 수도, 세 명일 수도, 아무도 없을 수도 있다(그 칸은 공격팀 영역이므로 아무에게도 안 간다).

3걸음 · 나누기

칸을 함께 덮은 수비수가 k 명이면 각자 $w(\text{칸}) \div k$ 씩 가져간다

협력게임 이론의 Shapley 값에서 **효율성**과 **대칭성**을 만족하는 배분이다.

· **효율성**: 한 프레임에서 모든 수비수가 받은 SC의 합 = 통제된 총 위험도 (코드가 매번 검사한다)

· **대칭성**: 같은 칸을 같은 조건으로 덮은 두 수비수는 같은 값을 받는다

덕분에 "혼자 넓게 막은 수비수"와 "둘이 겹쳐 막은 수비수"가 공평하게 평가된다.

프레임마다 위험도 총량이 크게 다르므로(공이 자기 박스 안이면 총량 320, 상대 박스 쪽이면 29로 11배 차이) 원값 SC와 함께 팀 총량 대비 몫인 `SC_share`도 낸다. 효율성 공리 덕분에 한 프레임의 share 합은 정확히 1이다.

10.5 PR — 압박 기여

목표물은 상황에 따라 다르다. 소유자가 정해진 프레임에서는 **볼 소유자**, 정해지지 않은 프레임에서는 **공**이다.

$$\text{근접도} = \text{clip}(1 - d \div 10m, 0, 1)$$

$$\text{PD (거리향)} = \text{근접도}^2 \quad \text{PV (속도향)} = \text{clip}(\text{접근속도} \div 5, 0, 1) \times \text{근접도}$$

$$\text{PR} = 0.6 \times \text{PD} + 0.4 \times \text{PV}$$

접근속도는 거리의 중앙차분에 음수를 붙인 값(가까워지면 +). 근접도를 제공하는 것은 '붙어 있는 것'에 확실히 더 큰 값을 주기 위해서다.

속도향에도 근접도를 곱하는 이유 — 고쳐진 설계 오류

원래 속도향에는 거리가 전혀 안 들어갔다. 그래서 공에서 40m 떨어진 수비수가 공 쪽으로 5 m/s로 달리지만 해도 $\text{PV}=1 \rightarrow \text{PR}=0.4$ 가 나왔고, 이는 소유자에게 1m까지 붙어 자리를 지킨 수비수($\text{PR}=0.486$)와 거의 같았다.

모사 경기에서 재 보니 **25m 밖에서 쌓인 값이 PR 총량의 50%**였고, 선언한 가중치는 60:40인데 실제 기여는 **거리 18 : 속도 82**였다. 먼 거리에서의 복귀 주력은 '압박'이 아니라 '활동량'이다.

상황	PD	PV	PR	해석
소유자에게 3m, 2 m/s로 접근	0.49	0.28	0.406	제대로 된 압박
소유자에게 1m, 정지(자리 지킴)	0.81	0.00	0.486	붙어서 막고 있음
30m 밖에서 5 m/s로 복귀	0.00	0.00	0.000	압박이 아니라 활동량

결과표에는 `PR_zone` (압박권 안에 있던 프레임 비율 = 관여의 양)과 `PR_in` (관여했을 때의 평균 PR = 관여의 질)이 따로 나온다. PR 평균 하나에는 이 둘이 섞여 있어 수비형 미드필더와 센터백이 구분되지 않기 때문이다.

10.6 PA — 패스 유인도

성분	정의	의미
l 길목 개방도	$\text{clip}(\text{패스선까지 최소 수직거리} \div 3\text{m}, 0, 1)$	소유자와 나를 잇는 직선을 수비수가 얼마나 막고 있다. 수선의 발이 선분 안에 있는 수비수만 센다 — 뒤쪽이나 너머의 수비수가 길을 막는 것으로 잘못 계산되는 것을 막는다.
ϕ 수신 공간	$\text{clip}(\text{최근접 수비수 거리} \div 8\text{m}, 0, 1)$	공을 받았을 때 자유로운가.
π 전진 가치	$0.3 + 0.7 \times (\text{clip}(\text{이득} \div 30\text{m}, -1, 1) + 1) \div 2$	이득 = (소유자의 골대까지 거리) - (내 골대까지 거리). 후방 패스도 0 은 아니므로 하한 0.3.

$$PA = l^a \times \phi^b \times \pi^c \quad (\text{기본 } a=b=c=1)$$

곱셈 모델이라 **하나라도 0 이면 전체가 0**. 아무리 자유롭고 전진하는 자리라도 패스길이 막혀 있으면 선택지가 아니라는 뜻이고, 축구의 상식과 맞는다.

예 — 길목 1.5m(=0.5), 최근접 수비수 4m(=0.5), 전진 이득 15m(=0.825) → PA = **0.206**

전진 가치를 X 좌표가 아니라 '골대까지 거리'로 재는 이유

X 성분만 보면 측면으로 벌리는 전환 패스나 대각 패스가 전부 '전진 0'으로 깎인다. 실제 빌드업의 상당 부분이 그 대각·측면 전환이고, 골문에서 먼 측면으로 가는 패스와 골문 정면으로 파고드는 패스는 가치가 분명히 다르다. 골대까지의 유클리드 거리 감소로 재면 두 경우가 자연스럽게 구분된다.

지수 학습 — 채택 조건이 엄격하다

곱셈 모델에 로그를 씌우면 선형모델이 되므로, 실제 패스를 정답 삼아 로지스틱 회귀로 지수를 추정할 수 있다. 양성은 "실제로 공을 받은 선수", 음성은 "같은 프레임의 나머지 아군"이다. 다만 채택하려면 넷을 다 넘어야 한다.

- 패스 표본 **200개 이상**
- 패스 단위 70/30 분할 후 **학습에 안 쓴 패스로** AUC 측정 (검증용 50개 미만이면 포기)
- **순열검정** — 정답을 무작위로 섞었을 때의 개선 분포를 60회 만들어 그 95% 지점을 넘어야 한다
- 길목 계수가 음수면(신호가 뒤집힘) 무조건 기각

이 장치가 막아 주는 것

예전 문턱은 30개였는데, "패스 상대가 무작위인" 모사 데이터에서 패스 44개로 AUC 0.387 → 0.613 이 나와 순열검정까지 넘고 채택된 적이 있다. **존재하지 않는 신호를 학습한 것이다**. 한 번 채택되면 모든 PA 값이 그 지수로 다시 계산되므로 TC 순위까지 잡음이 된다.

10.7 합성 — DPI · TC · TC_adj

$$DPI = 0.5 \cdot z(SC) + 0.5 \cdot z(PR)$$

$$TC = \frac{1}{3} \cdot n(SC) + \frac{1}{3} \cdot n(PR) + \frac{1}{3} \cdot n(PA) \quad n() = \text{순위 정규화}$$

z 가 아니라 순위 기반을 기본으로 쓰는 이유: SC 는 좌우대칭인데 PR 은 오른쪽으로 길게 늘어져 있다(왜도 +1.2, 최대/중앙 6.5). z 로 더하면 꼬리가 긴 PR 의 극단값이 합계를 끌고 가서 가중치는 1/3씩인데 실제 영향력은 그렇지 않게 된다.

합치기 전에 빼는 것

- **골키퍼** — 위험도가 최대인 골문 앞에 상시 서 있어 가만히 있어도 SC 를 쓸어담는다(모사 경기에서 SC 3위/7위). 자기 골대까지 평균거리 22m 미만인 '팀당 한 명'을 골키퍼로 본다(실측: 골키퍼 ~20m, 센터백 ~31m). **검출기가 골키퍼를 알려주면 그것을 우선한다** — tracks.csv 의 `class_name` 이 여기 쓰인다.
- **표본이 너무 적은 선수** — 팀 중앙값의 20% 미만. 평균이 튀는 것도 문제지만, 더 큰 문제는 그 값이 **정규화** 기준까지 흔들려 남의 점수를 바꾼다는 것이다.

TC_adj — 포지션 보정

TC 는 '어느 자리에서 뛰는가'와 상관이 크다. 실측 순위상관은 SC 가 $\rho = -0.87$ (뒤에 설수록 높음), PR 이 +0.68, PA 가 +0.29 다. 그래서 TC 를 '자기 골대까지 평균거리'로 회귀한 **잔차**를 함께 낸다.

두 숫자를 읽는 법

TC = 팀에 준 총량 · **TC_adj** = 같은 위치의 선수들과 비교했을 때

둘 중 하나가 정답이 아니다. **둘이 많이 다르면 그 순위는 포지션이 만든 것**이라는 뜻으로 읽는다. 코드는 TC 의 몇 %가 거리 하나로 설명되는지(R^2)를 매번 출력하고 40%를 넘으면 경고한다.

가중치에 대한 태도

0.5/0.5 와 1/3씩은 모두 "검증 전 기준선"이다. 어느 하나가 더 중요하다는 선험적 근거가 없고, **기존 선수 순위에 맞춰 가중치를 조정하면 결론을 먼저 정해 놓고 식을 맞추는 것**이 되기 때문이다. 대신 실행할 때마다 "가중치를 바꿨을 때 순위가 얼마나 흔들리는지"를 출력한다.

10.8 진단 출력 읽는 법

구간	확인할 것
[검증]	12항목 모두 PASS 인가. 하나라도 FAIL 이면 아래 숫자는 볼 필요가 없다.
[읽기]	좌표 범위가 경기장과 맞는가 · 공격 방향 판정이 맞는가 · 프레임당 인원이 명목의 90% 이상인가 · ID 재연결 후 ID 개수가 선수 수와 비슷한가.
[계산]	usable 비율 . 30% 미만이면 원인이 '소유자 미판정'인지 '추적 인원 부족'인지도 함께 나온다. 앞쪽이 크면 공 검출 문제, 뒤쪽이 크면 화면 밖 선수 문제다.

[지표 진단]	바닥 효과 (대부분 0)와 포화 (대부분 1)를 본다. 포화가 절반을 넘으면 그 차원은 순위에 기여하지 못한다. 분할 재현성 (앞구간 vs 뒷구간 순위상관)도 낸다 — 낮으면 선수가 아니라 상황을 재고 있는 것이다.
[총 기여도]	무엇을 뺐는지, TC 의 몇 %가 포지션인지, 가중치를 바꾸면 순위가 얼마나 흔들리는지.
표본검사 시트	선수마다 점수를 가장 많이 끌어올린 프레임의 영상 시각(mm:ss) 을 뽑아 준다. 최소 5초 간격으로 골라 서로 다른 장면이 나오게 한다. 영상에서 그 지점 몇 개만 열어 보면 표 전체를 믿어도 되는지 금방 알 수 있다 — draw_boxes.py 와 같은 철학이다.

11 장

두 단계 사이 — 끊어져 있는 여섯 곳

여기부터는 코드를 읽기만 한 것이 아니라 **실제로 실행해서 확인**한 내용이다. tracker.py 가 내는 CSV 열 목록을 기여도 코드의 열 인식 함수에 그대로 넣어 보고, 공 판정 함수에 실제 값을 넣어 봤다.

11.1 좌표가 픽셀이다 가장 심각

CSV 의 `fx`, `fy` 주석과 README 모두 "지면 좌표로 바꿀 때 이 값을 쓴다"고 적혀 있다. 그런데 **변환 자체가 구현되어 있지 않다**. tracker.py 2,593행 어디에도 homography · perspective · warp · 미터 변환이 없다.

기여도 코드는 픽셀을 받으면 스스로 멈춘다.

좌표 범위가 경기장보다 훨씬 크다 (X 0~1920, Y 0~1080).
화면 픽셀 좌표로 보인다. 호모그래피로 미터 좌표로 변환한 뒤에 넣어야 한다.
픽셀로 계산하면 화면 위/아래의 1m 가 서로 달라져서 결과가 전부 무의미해진다.

왜 픽셀로는 절대 안 되는가

카메라는 경기장을 비스듬히 본다. 그래서 **화면 아래쪽(가까운 곳)의 100픽셀과 위쪽(먼 곳)의 100픽셀이 서로 다른 거리다**. 가까운 쪽에서 5m 인 간격이 먼 쪽에서는 20m 일 수 있다.

기여도 계산의 거의 모든 상수가 미터 기준이다 — 격자 1m, 위험도 감쇠 18m·20m, 압박권 10m, 패스 길목 3m, 수신 공간 8m, 소유 반경 2~8m, 최고속도 11 m/s. 픽셀을 넣으면 이 상수들이 화면 위치에 따라 제멋대로 늘어났다 줄었다 한다. **계산은 조용히 끝나지만 숫자는 전부 무의미하다**.

11.2 열 이름이 맞지 않는다 실행 확인

tracker.py 의 `CSV_COLUMNS` 를 기여도 코드의 `map_columns()` 에 넣고 돌린 결과다.

```
열 인식: frame ← 'frame' , time_s ← 'time_sec' , track_id ← 'track_id' ,  
        conf ← 'conf' , team ← 'team'
```

```
인식된 표준 열 : ['frame', 'time_s', 'track_id', 'conf', 'team']  
필수 열 중 없는 것 : {'X', 'Y'} ← 여기서 멈춘다
```

기여도 코드의 별칭 목록에 `x`, `pos_x`, `pitch_x` 는 있지만 `cx` · `fx` 는 없다. `class_name` · `class_id` 도 `cls` 로 인식되지 않아, **심판 제외와 골키퍼 자동 지정이 작동하지 않는다**. 변환 단계에서 이름만 바꿔 주면 해결된다.

11.3 팀 번호 규약이 다르다

tracker 는 0 / 1 / -1 / 빈칸(공)을 쓴다. 기여도 코드는 팀이 정확히 2개여야 하고, 심판으로 인식하는 키워드는 `ref`, `심판`, `none`, `nan`, `unknown` 이다. -1 은 이 목록에 없으므로 **세 번째 팀**으로 세어지고 다음에서 멈춘다.

팀이 2개여야 하는데 ['0', '1', '-1'] 이다. team 열을 정리할 것.

단순히 -1 을 다 빼면 안 된다

축구 전용 모델은 골키퍼도 `team=-1` 로 표시한다(팀 색 군집에서 빠지므로). -1 을 통째로 빼면 골키퍼까지 사라진다. 그러면 골문 앞 공간을 센터백이 대신 받아 SC 가 부풀려지고, 기여도 코드의 골키퍼 제외 로직도 대상을 못 찾는다.

올바른 처리: `class_name == 'referee'` 인 행만 빼고, `goalkeeper` 는 남긴다. 그리고 그 `track_id` 목록을 기여도 코드의 `GK_IDS` 에 직접 넣는다(자동 추정보다 정확하다).

11.4 공 행 인식 지금 설정에서는 작동

기여도 코드의 공 판정 함수에 실제 값을 넣어 본 결과다.

넣은 값	공으로 인식?	어떤 경우인가
'ball'	✓ 인식	축구 전용 모델의 클래스 이름
1000001	✓ 인식	공 전용 모델이 붙인 track_id
'sports ball'	✗ 못 함	COCO 모델의 클래스 이름 — 정확히 'ball' 이어야 인식
32	✗ 못 함	COCO 공 클래스 번호

다행인 우연

tracker.py 의 `ball_id_base = 1_000_000` 과 기여도 코드의 `BALL_ID_BASE = 1_000_000` 이 값이 같다. 지금 설정은 공 전용 모델을 쓰므로 **track_id** 만으로 공 행이 정확히 인식된다. 공 전용 모델을 끄면 이 통로가 막혀 "공 좌표를 찾지 못했다"로 중단된다.

11.5 interpolated 열이 무시된다 경미

tracker 는 보간으로 메운 공 위치를 `interpolated=1` , `conf=0` 으로 정직하게 표시한다. 그런데 기여도 코드는 그 열을 읽지 않고 자기만의 보간을 따로 한다. 결과적으로 **추정 위치와 측정 위치가 똑같은 자격으로 소유자 판정에 쓰인다**. 치명적이지는 않지만, 어느 기여가 실측에서 나왔는지 구분할 수단이 사라진다. `conf` 열은 인식되므로, 기여도 코드의 `BALL_MIN_CONF` 를 0보다 크게 두면 보간 행(`conf=0`)만 자연히 빠진다.

11.6 공 높이 정보가 없다 경미

기여도 코드에는 공중볼(높이 1.5m 초과) 구간을 소유자 판정에서 빼는 장치가 있는데, `tracks.csv` 에는 높이 열이 없어 **작동하지 않는다**. 크로스-헤더 경합 구간에서 지면상 가장 가까운 선수가 소유자로 잡힌다. 단안 카메라로 공 높이를 추정하는 것은 별도 연구 주제이므로, 지금은 "그런 한계가 있다"로 알아 두면 된다.

12 장

사례 해부 — 결과표가 망가진 이유

앞서 보여주신 결과표에는 세 가지 증상이 있었다. 셋 다 같은 뿌리에서 나온다.

증상 ① — 선수가 22명인데 표에 160행이 나왔다

track_id	team	frames	SC_mean	PR_mean	PA_mean	TC
Red_new102_12	Red	13	1.6588	0.0000	0.6510	0.1690
Red_new102_8	Red	13	1.0451	0.2507	0.1722	0.0590
Blue_new193_9	Blue	1	NaN	NaN	0.0181	NaN
...						(160행 계속)

Red_new102_12 는 기여도 코드가 붙인 이름으로, 팀_new{프레임}_{번호} 형식이다 — "102번 프레임에서 새로 생긴 12번 궤적". 한 선수의 궤적이 토막나 있다는 뜻이다.

원인 1 · ByteTrack 이 주는 track_id 가 오래 못 간다

tracker.py 주석의 실측: 720p · 프레임간격 3 에서

트랙 지속 중앙값 2장(0.2초)

, match_thresh 를 0.9 로 올려도 7→13장(0.5초). 선수 한 명이 20초 동안 수십 개 번호를 갈아탄다.

지금 config.json 이 정확히 그 조건(stride 3 · imgsz 640)이다.

원인 2 · 기여도 코드의 ID 재연결이 '새 번호 발급'으로 대응한다

헝가리안 매칭으로 앞 프레임과 이어 붙이되, 이어지지 않는 검출에는 새 이름을 준다.

한 번 끊긴 궤적은 다시 만나지 않는다

— 몇 프레임 뒤 같은 자리에 나타나도 별개의 선수가 된다.

결과 · 22명이 160개 ID 로 부서진다

같은 조건을 모사 데이터로 재현하니 22명 →

948개 ID

까지 부서졌다.

이것은 수식의 문제가 아니다

SC · PR · PA 자체는 정상 작동했고 효율성 공리 검산도 통과했다. 무너진 것은 "같은 선수"라는 개념이다. 그래서 프레임 단위 결과(SC/PR/PA 분포)까지는 의미가 있어도 선수 순위는 의미가 없다. 기여도 코드도 이것을 알고 경고를 출력한다.

증상 ② — TC 가 대부분 비어 있다(NaN)

1. 구조적 이유 — Red 쪽에는 SC·PR 은 있는데 PA 가 없고, Blue 쪽에는 PA 만 있다. 관측 구간 내내 Blue 가 공을 갖고 Red 가 수비했다는 뜻이다. TC 는 세 지표가 다 있어야 계산되므로 양쪽 다 낼 수 없다. 더 길고, 공수가 오가는 구간이 필요하다.

2. **파편화 이유** — 1~2프레임만 존재하는 조각은 셋 중 하나만 걸리기 쉽다. 160행 중 대부분이 `frames=1` 이었다.

증상 ③ — PR 이 거의 전부 0.0000 이다

PR 압박권($d \leq 10m$) 안 0.0% / 공까지 거리 중앙값 50.1 m

공까지 거리 중앙값 50m 는 축구 경기에서 나올 수 없는 값이다. 압박을 안 했다는 뜻이 아니라 **좌표계가 틀렸다는 신호**다 — 11.1 의 픽셀/미터 문제이거나, 공 좌표와 선수 좌표의 정합이 깨진 경우다.

함께 나온 신호들

출력	의미
usable 4%	96%의 프레임이 계산에서 빠졌다. '소유자 미판정 60% / 추적 인원 부족 91%' 로 원인이 함께 찍혔고, 뒤쪽이 크므로 화면 밖 선수 가 주원인이다.
프레임당 인원이 명목의 73%	22명 중 16명만 잡혔다. 한 대의 카메라로는 경기장의 2/3 만 들어온다.
골키퍼를 1명만 찾았다	ID 파편화로 '자기 골대 앞에 상주하는 한 명'을 찾을 수 없었다. 남은 골키퍼가 SC 상위권을 차지한 채 순위에 들어간다.
SC_mean 24.5 (<code>frames=6</code>)	6프레임만 존재한 조각이 SC 1위다. 수비수가 몇 명 안 잡힌 프레임에서 그 조각이 빈 자리의 공간 점유까지 받아간 결과다.

13 장

고치는 순서

네 가지를 이 순서로 해야 한다. 앞의 것을 안 고치고 뒤의 것을 고치면 헛수고가 된다.

1순위 · 픽셀 → 미터 변환 (호모그래피)

이것 없이는 나머지가 전부 무의미하다. 경기장의 **알려진 지점 4개 이상**을 화면에서 찍어 두 평면 사이의 변환 행렬을 구한다.

```
import cv2, numpy as np, pandas as pd

# 화면에서 찍은 점 (픽셀)                실제 경기장 위치 (미터)
src = np.float32([[412, 880], [1503, 872], [1740, 402], [188, 410]])
dst = np.float32([[ 0, 0], [ 105, 0], [ 105, 68], [ 0, 68]])
H, _ = cv2.findHomography(src, dst)

df = pd.read_csv("results/경기/tracks.csv")
pts = np.float32(df[["fx", "fy"]].values).reshape(-1, 1, 2)
xy = cv2.perspectiveTransform(pts, H).reshape(-1, 2)
df["X"], df["Y"] = xy[:, 0], xy[:, 1]
```

쓸 만한 기준점: 경기장 네 모서리, 페널티박스 네 모서리, 골에어리어 모서리, 하프라인과 터치라인이 만나는 두 점, 센터서클 좌우 끝.

중계 영상이면 이것만으로 부족하다 — 제작자도 같은 결론을 적어 두었다

위 방법은 **카메라가 고정일** 때만 한 번으로 끝난다. 중계 카메라는 패닝·줌·틸트를 하므로 **프레임마다 변환 행렬이 달라진다**. 선택지는 셋이다.

- ① 고정 전술 카메라 영상을 쓴다 — 가장 쉽고 확실하다
- ② 프레임마다 경기장 선을 검출해 변환을 다시 계산한다(허프 변환 등)
- ③ 경기장 키포인트를 예측하는 모델을 붙인다(SoccerNet 계열 공개 모델)

README 6장 '알려진 한계'에 "**전술 카메라 기준으로 맞췄다. 일반 중계 화면에서는 리플레이·클로즈업 장면에서 결과가 흔들린다**"고 이미 적혀 있다. 같은 제약에서 나온 같은 결론이다. 기여도 계산은 어차피 경기장 전체가 보이는 좌표를 요구한다(12장의 usable 4%).

2순위 · 추적 지속 늘리기 — 목표는 "한 선수 = 한 ID"

바꿀 것	현재 → 권장	이유
<code>vid_stride</code>	3 → 2 또는 1	파편화의 최대 원인. README 자신이 "선수 번호가 자주 바뀌면 '보통'(=2)을 쓰라"고 권한다.
<code>imgsz</code>	640 → 960	박스가 커지면 검침 계산이 안정된다. 검출 수는 거의 안 늘지만 추적은 좋아진다.

<code>use_opencvino</code>	false → true	GPU 가 없다면 위 두 변경의 비용을 상쇄해 준다(30분 영상 20.3 → 12.8분).
<code>track_buffer</code>	30 → 45~60	지금은 원본 기준 약 1.2초. 겹침으로 잠깐 사라진 선수를 더 오래 기억한다.
<code>bytetrack_recover_low</code>	false → true	낮은 신뢰도 검출을 트랙 잇기 용도로만 쓴다. 가려졌다 나온 선수의 ID 가 덜 끊긴다.

확인 방법 — 20초만 돌려 보면 된다

`python tracker.py 경기.mp4 --from 12:00 --to 12:20` 으로 짧게 돌린 뒤 `summary.json` 의 `unique_track_ids` 를 본다.

- 30 안쪽 → 성공. 기여도 계산으로 넘어가도 된다
- 100 이상 → 아직 파편화. 설정을 더 조인다

`python draw_boxes.py 경기 --from 12:00 --to 12:20 --labels id` 로 영상을 만들어 번호가 얼마나 자주 바뀌는지 눈으로 보는 것이 가장 확실하다.

3순위 · 변환 스크립트 (tracks.csv → 기여도 입력)

11장의 나머지를 한 번에 처리하는 30~40줄짜리 스크립트다. 하는 일은 여섯 가지다.

1. 공 행(`track_id ≥ 1,000,000` 또는 `class_name == 'ball'`)을 떼어 `ball_x`, `ball_y` 로 만든다 — 공은 `cx`, `cy` 를 쓴다
2. 사람 행에서 `class_name == 'referee'` 만 뺀다 (골키퍼는 남긴다)
3. `fx`, `fy` 에 호모그래피를 적용해 `x`, `y` (미터)를 만든다
4. `team` 0/1 을 알아보기 쉬운 이름으로 바꾼다 (예: Home / Away)
5. 골키퍼 `track_id` 목록을 뽑아 기여도 코드의 `GK_IDS` 에 넣는다
6. 보간 행을 뺄 생각이면 `interpolated == 1` 을 거르거나 기여도 코드의 `BALL_MIN_CONF` 를 0.01 이상으로 둔다

4순위 · 기여도 코드 쪽 정리

- **꺾적 잇기(track stitching)** — 시간이 겹치지 않으면서 끝점과 시작점이 시공간적으로 이어지는 조각을 하나로 합친다. 1~3순위를 해도 완벽한 추적은 없으므로, 남은 파편을 해석 단계에서 흡수하는 장치다. 지금 코드에는 없다.
- **표 정리** — 관측이 너무 적은 조각은 화면 표에서 빼고(엑셀에는 남기고), NaN 을 - 로 표시하고, 22행 남짓만 보이게 한다.
- **TC 를 부분 지표로도 계산** — 세 지표가 다 없으면 지금은 NaN 인데, 있는 것끼리 가중치를 다시 나눠 계산하고 "무엇으로 계산했는지"를 함께 표시하면 짧은 클립에서도 쓸 수 있다.

14 장

라이선스와 알려진 한계

14.1 라이선스 — AGPL-3.0 주의

구성요소	라이선스	출처
Ultralytics YOLO11	AGPL-3.0	ultralytics/ultralytics
선수 모델 soccer_yolo11m.pt	AGPL-3.0	martinjolif/yolo-football-player-detection (HuggingFace)
공 모델 soccer_ball_yolo11n.pt	AGPL-3.0	martinjolif/yolo-football-ball-detection (HuggingFace)

무엇이 되고 무엇이 안 되나

괜찮다 — 개인·연구용 분석, 논문·보고서에 결과를 실는 것, 학교 과제·내부 검토.

의무가 생긴다 — 이 프로그램을 수정해서 배포하거나 웹 서비스로 제공하면 AGPL-3.0 에 따라 소스 코드를 공개해야 한다. AGPL 은 GPL 과 달리 "네트워크 너머로 서비스만 제공해도" 공개 의무가 발생한다.

기여도 분석을 서비스화할 계획이 있다면 미리 짚어야 할 지점이다. 피하려면 AGPL 이 아닌 검출 모델로 바꾸거나 (라이선스가 다른 모델을 직접 학습), Ultralytics 의 상용 라이선스를 구매하는 선택지가 있다.

14.2 제작자가 밝힌 한계 (README 6장)

한계	이 문서의 대응
전술 카메라 기준으로 맞췄다. 일반 중계 화면에서는 리플레이·클로즈업에서 결과가 흔들린다	13장 1순위와 정확히 같은 결론. 호모그래피도 고정 카메라를 전제해야 쉽다.
공 발견률은 전술 카메라 1080p 기준 약 75~80%	기여도의 소유자 판정이 그만큼만 가능하다는 뜻. usable 비율의 상한이 여기서 정해진다.
골키퍼는 영상에서 GK 로 통일되지만 CSV 의 track_id 는 가려졌다 나오면 바뀔 수 있다	기여도 코드의 골키퍼 자동 추정이 실패하는 원인. GK_IDS 직접 지정으로 대응한다.
CPU만으로는 느리다. 전체 경기는 밤새 돌리는 용도	실측 CPU 8코어 1080p '빠르게' = 영상 길이의 약 5배(45분 경기 ≈ 4시간).
관중석이 초록인 경기에서는 잔디 구분이 안 돼 경기장 밖 인물이 덜 걸러진다	해가 없는 쪽으로 실패한다(아무것도 안 거른다). 다만 기여도의 SC 는 영향을 받는다.

14.3 이 문서가 덧붙이는 한계

- **픽셀 좌표** — 지면 좌표 변환이 미구현(11.1). 기여도 계산의 전제가 성립하지 않는다.
- **공 높이 없음** — 크로스-헤더 경합에서 소유자 판정이 들어진다(11.6).
- **ID 지속** — 현재 설정에서 선수별 집계가 성립하지 않는다(12장).
- **한 화면의 한계** — 카메라 한 대로는 경기장의 2/3 만 담긴다. SC 는 경기장 전체 격자를 나누는 계산이라 이 부분이 구조적 제약이다.

15 장

용어 사전

용어	뜻
검출 (Detection)	한 장의 이미지에서 "여기에 사람이 있다"를 찾는 것. 프레임 사이의 관계는 모른다.
추적 (Tracking)	프레임을 가로질러 "이 사람과 저 사람이 같은 사람"임을 잇는 것. 그 결과가 track_id.
ByteTrack	검출 결과를 잇는 추적 알고리즘. 칼만 필터로 다음 위치를 예측하고 박스 겹침(IoU)으로 짝짓는다. 낮은 신뢰도 검출도 2단계로 활용하는 것이 특징이다.
IoU	두 박스의 겹친 넓이 ÷ 합친 넓이. 1이면 완전히 같고 0이면 안 겹친다.
헝가리안 매칭	N개와 M개를 짝지을 때 전체 비용 합이 최소가 되는 짝을 찾는 알고리즘. lap 이 이것을 한다.
칼만 필터	지금까지의 움직임으로 다음 위치를 예측하는 방법. 검출이 잠깐 없어도 추적을 이어 준다.
호모그래피	한 평면(화면)의 점을 다른 평면(경기장)으로 옮기는 3×3 변환 행렬. 기준점 4개면 구할 수 있다.
HSV / Lab	색을 표현하는 좌표계. HSV 는 색상·채도·명도(잔디 찾기에 씬), Lab 은 밝기(L)와 색(a·b)이 분리되어 그들의 영향을 뺄 수 있다(팀 구분에 씬).
k-means	점들을 k 개 덩어리로 묶는 군집 알고리즘. 여기서는 상체 색을 4덩어리로 묶어 두 팀을 찾는다.
모폴로지 연산	마스크의 구멍을 메우거나(CLOSE) 잔점을 지우는(OPEN) 영상 처리 연산.
Shapley 값	여럿이 협력해 만든 성과를 각자의 기여에 따라 나누는 협력게임 이론의 배분 방식. 합이 보존되고 (효율성) 같은 기여는 같은 몫을 받는다(대칭성).
z 점수 / 순위 정규화	단위가 다른 값을 비교 가능하게 만드는 방법. z 는 (값-평균)÷표준편차, 순위 정규화는 순위를 정규분포 점수로 바꾼다. 후자는 분포 모양에 안 휘돌린다.
AUC	어떤 점수가 두 부류를 얼마나 잘 가르는지를 0.5~1 로 나타낸 값. 0.5는 동전 던지기, 1은 완벽.
순열검정	정답을 무작위로 섞은 가짜 데이터에서도 같은 성능이 나오는지 확인하는 검정. "우연히 좋아 보이는 것"을 걸러낸다.
OpenVINO	인텔이 만든 CPU 추론 가속 라이브러리. 모델을 한 번 변환해 두면 1.5~2배 빨라진다.
usable	기여도 계산에 실제로 쓰인 프레임. 소유팀이 정해졌고 양 팀 인원이 충분히 추적된 프레임.

부록

표 모음

A. tracks.csv 열 — 9.1 참고

B. summary.json 주요 필드 — 9.2 참고

C. 상수 전체표 (내 설정값 기준)

C-1. 측정 단계 (tracker.py / config.json)

상수	내 값	뜻과 근거
model	soccer_yolo11m	선수·골키퍼·심판·공 4클래스. COCO 대비 프레임당 12.1 → 23.1명
imgsz / conf	640 / 0.10	검출 해상도 / 신뢰도 하한
vid_stride	3	3프레임에 1장. 추적 파편화의 최대 원인
match_thresh	0.9	트랙 연결 허용 상한. 0.8→0.9 로 ID 233→139개
track_buffer	30 (→10 환산)	가려진 선수 기억. stride 로 나눠 원본 기준 유지
pitch_hue / pitch_shrink	30~95 / 16px	잔디 HSV 범위 / 가장자리 수축
static_max_move	0.10	(이동폭/키) 기준. 축구 모델에서는 미적용
ball_ratio_min/max	0.16 / 0.36	공 크기 ÷ 사람 키. 진짜 공 0.170~0.319
ball_min_bright	215	흰 덩어리 최대 밝기. 공 248 · 마크 196
ball_min_fill	0.65	채움 비율. 공 0.77(원 0.785) · 축구화 0.52
ball_imgsz / ball_ratio_scale	1920 / 0.89	공 해상도 / 크기 보정. 반드시 짝으로 바꾼다
ball_focus_size/imgsz	480 / 960	주시 확대. 신뢰도 0.04 → 0.7~0.8
ball_reach / ball_switch_margin	140px / 0.20	연속성 감쇠 거리 / 갈아타기 문턱
ball_chain_tol / _min_len	28px / 3	궤적 잇기 허용 오차 / 최소 길이
interp_max_gap_sec	0.4초	보간 상한. 0.8초면 오차가 선수 키의 0.28
keeper_track_conf	0.50	트랙 신뢰도 중앙값. 골대 0.22~0.41 · 골키퍼 0.60~0.81

<code>team_clusters /</code> <code>team_samples</code>	4 / 16	군집 수 / 표본 프레임 수
<code>team_merge_ratio /</code> <code>team_imgsz</code>	0.5 / 960	군집 병합 거리 / 팀 색 전용 해상도
<code>range_warmup_sec</code>	4.0초	구간 분석 시 미리 돌리는 시간
<code>ball_id_base</code>	1,000,000	공 track_id 시작 번호. 기여도 코드와 값이 일치

C-2. 해석 단계 (soccer_dpi_pa_v7.py)

상수	값	뜻과 근거
<code>GRID_STEP</code>	1.0 m	SC 격자. 0.5 로 하면 90분에 2시간 이상
<code>LAMBDA_GOAL / BALL</code>	18 / 20 m	위험도 감쇠 거리(골대 / 공)
<code>PR_R / PR_VMAX</code>	10 m / 5 m/s	압박권 반경 / 접근속도 정규화 기준
<code>PR_WD : PR_WV</code>	0.6 : 0.4	PR 내부 가중치(거리 : 속도)
<code>W_LANE / PA_SPACE_R</code>	3 m / 8 m	패스 길목 폭 / 수신 공간 만점 거리
<code>PA_FWD_REF / PA_PI_MIN</code>	30 m / 0.3	전진 이득 기준 / 전진 가치 하한
<code>PA_MIN_PASSES</code>	200	지수 학습 최소 표본. 44개로 가짜 신호가 채택된 적이 있다
<code>SPEED_MAX</code>	11 m/s	= 39.6 km/h. 프로 실측 최고 34~37 km/h
<code>POSS_RADIUS /</code> <code>HOLD_FRAMES</code>	자동 2~8 m / 1.5초	공 소유 인정 거리 / 전환 유지 시간
<code>GK_MAX_DIST</code>	22 m	골키퍼 판정. 골키퍼 ~20m · 센터백 ~31m
<code>W_SC : W_PR / TC 가중치</code>	0.5:0.5 / ⅓씩	검증 전 기준선. 순위에 맞춰 조정하면 안 된다
<code>TC_NORM</code>	rank	순위 정규화. PR 의 긴 꼬리가 합계를 지배하는 것을 막는다

이 문서의 모든 수치는 두 가지 출처 중 하나다 — (1) 코드 주석에 기록된 실측값, (2) 문서를 쓰면서 코드를 직접 실행해 확인한 값(11.2 · 11.4 · 12장의 재현 실험). 추측으로 채운 숫자는 없다. 1판에서 잘못 쓴 두 곳은 8.4 에 정정해 두었다.